テスト技術者資格制度

Advanced Level シラバス日本語版
テストアナリスト (CTAL-TA)

Version 4.0.J01

International Software Testing Qualifications Board



著作権

Copyright Notice © International Software Testing Qualifications Board（以降は ISTQB®）

ISTQB® は、International Software Testing Qualifications Board の登録商標である。

Copyright © 2025 The authors of v4.0: Armin Born, Filipe Carlos, Wim Decoutere, István Forgács, Matthias Hamburg (Product Owner), Attila Kovács, Sandy Liu, François Martin, Stuart Reid, Adam Roman, Jan Sabak, Murian Song, Tanja Tremmel, Marc-Florian Wendland, Tao Xian Feng

Copyright © 2021-2022. The authors of the update v3.1.0, errata 3.1.1, and errata 3.1.2: Wim Decoutere, István Forgács, Matthias Hamburg, Adam Roman, Jan Sabak, Marc-Florian Wendland.

Copyright © 2019 The authors of the update 2019: Graham Bath, Judy McKay, Jan Sabak, Erik van Veenendaal.

Copyright © 2012. The authors: Judy McKay, Mike Smith, Erik van Veenendaal.

無断転載を禁じる。ここに、本書の筆者グループは、著作権をInternational Software Testing Qualifications Board（ISTQB®）に譲渡する。本書の筆者グループ（現在の著作権保持者）とISTQB®（将来の著作権保持者）は、以下の使用条件に合意している。

- 出典を明記する場合に限り、非営利目的で本書の一部を抜粋し、複製することができる。認定教育機関は、著者およびISTQB®をシラバスの出典および著作権所有者として明記し、かつ、ISTQB®が承認する各国委員会からトレーニング教材の公式な認定を受けた後にのみ、トレーニングコースの広告で本シラバスに言及することを条件として、本シラバスをトレーニングコースのベースとして使用することができる。
- 著作者やISTQB®が本シラバスの出典および著作権の保有者であることを明記する限りにおいて、個人または個人のグループが本シラバスを記事、書籍のベースとして使用してもよい。
- ISTQB®の書面による承認を得ることなく、本シラバスを他の目的で使用することは禁止する。
- ISTQB®が承認する各国委員会は、シラバスの翻訳版に上記の著作権表示を転載することを条件に、本シラバスを翻訳することができる。

Translation Copyright © 2005-2026, Japan Software Testing Qualifications Board (JSTQB®), all rights reserved. 日本語翻訳版の著作権はJSTQB®が有するものです。本書の全部、または一部を無断で複製し利用することは、著作権法の例外を除き、禁じられています。

改訂履歴

ISTQB®

バージョン	日付	摘要
v4.0	2025/05/02	全体的な改訂と範囲の更新を伴うメジャーアップデート
v3.1.2	2022/01/31	正誤表:マイナーな書式、文法、表現の欠陥の修正
v3.1.1	2021/05/15	正誤表:著作権表示を現在のISTQB®標準に適合
v3.1	2021/03/03	3.2.3項の書き直しとさまざまな表現改善によるマイナーアップデート
v3.0 (2019)	2019/10/19	全体的な改訂とスコープの縮小に伴うメジャーアップデート
v2.0 (2012)	2012/10/19	別冊CTAL-TAシラバスとしての初版

JSTQB®

バージョン	日付	摘要
Version 4.0.J01	2026年6月9日	ISTQB Advanced Level Syllabus Test Analyst v4.0 の日本語翻訳版
Version 3.1.1.J02	2021年8月10日	「2.4.2 将来のテストサイクルに向けたテストの調整」において、記載内容を追加
Version 3.1.1.J01	2021年7月14日	ISTQB Advanced Level Syllabus Test Analyst v3.1.1 の日本語翻訳版
Version 2012.J01	2015年3月25日	ISTQB Advanced Level Syllabus Test Analyst Version 2012 の日本語翻訳版

©International Software Testing Qualifications Board

目次

著作権 *2*
改訂履歴 *3*
謝辞 *7*
0 イントロダクション *8*
0.1 本シラバスの目的 8
0.2 テスト技術者資格制度Advanced Level Test Analyst 8
0.3 テスト担当者のキャリアパス 8
0.4 ビジネス成果 10
0.5 学習の目的と知識レベル 10
0.6 Advanced Level Test Analystの資格認定試験 10
0.7 認定審査 11
0.8 標準の取り扱い 11
0.9 詳細レベル 11
0.10 本シラバスの構成 12
1 テストプロセスにおけるテストアナリストのタスク - 225分 *13*
テストプロセスにおけるテストアナリストのタスクについて 14
1.1 ソフトウェア開発ライフサイクルにおけるテスト 14
1.1.1 さまざまなソフトウェア開発ライフサイクルにおけるテストアナリストの関与 14
1.2 テスト活動への関与 14
1.2.1 テスト分析 15
1.2.2 テスト設計 15
1.2.3 テスト実装 16
1.2.4 テスト実行 16
1.3 作業成果物に関するタスク 17
1.3.1 ハイレベルテストケースとローレベルテストケース 17
1.3.2 テストケースの品質基準 18
1.3.3 テスト環境要件 18
1.3.4 テストオラクルの決定 19
1.3.5 テストデータ要件 20
1.3.6 キーワード駆動テストを使用したテストスクリプトの開発 20
1.3.7 テストウェアのマネジメントに適用するツール 21

©International Software Testing Qualifications Board

2 リスクベースドテストにおけるテストアナリストのタスク - 90分 23
リスクベースドテストにおけるテストアナリストのタスクについて 24
2.1 リスク分析 24
2.2 リスクコントロール 25
3 テスト分析とテスト設計 - 615分 27
テスト分析とテスト設計について 28
3.1 データに基づくテスト技法 28
3.1.1 ドメインテスト 28
3.1.2 組み合わせテスト 29
3.1.3 ランダムテスト 30
3.2 振る舞いに基づくテスト技法 30
3.2.1 CRUDテスト 31
3.2.2 状態遷移テスト 31
3.2.3 シナリオベースドテスト 32
3.3 ルールに基づくテスト技法 33
3.3.1 デシジョンテーブルテスト 33
3.3.2 メタモルフィックテスト 34
3.4 経験ベースのテスト 35
3.4.1 セッションベースドテストを支援するテストチャーター 35
3.4.2 経験ベースのテスト技法を支援するチェックリスト 36
3.4.3 クラウドテスト 37
3.5最も適切なテスト技法の適用 37
3.5.1 プロダクトリスクを軽減するテスト技法の選択 38
3.5.2 テスト設計自動化のメリットとリスク 39
4 品質特性のテスト - 60分 40
品質特性のテストについて 41
4.1 機能テスト 41
4.2 使用性テスト 42
4.3 柔軟性テスト 43
4.4 互換性テスト 43
5 ソフトウェア欠陥予防 - 225分 45
ソフトウェア欠陥予防について 46
5.1 欠陥予防の実践 46
5.2 フェーズ内封じ込めの支援 47
5.2.1 欠陥検出でのモデルの利用 47
5.2.2 レビュー技法の応用 48

©International Software Testing Qualifications Board

5.3 類似欠陥の再発の軽減 49
5.3.1 欠陥検出を改善するためのテスト結果の分析 49
5.3.2 欠陥分類による根本原因分析の支援 50
6 参考文献 52
標準 52
ISTQB® ドキュメント 52
書籍 53
記事 53
Webページ 55
7 付録A - 学習の目的／認知レベル 57
8 付録B-学習の目的とビジネス成果のトレーサビリティマトリクス 59
9 付録C - リリースノート 64
10 付録D - 略語リスト 66
11 付録E - 特定の専門分野（ドメイン）特有の用語 67
12 付録F - ソフトウェア品質モデル 68
13 付録G - 商標 70

©International Software Testing Qualifications Board

謝辞

このドキュメントは、2025年5月2日に開催されたISTQB®総会で正式に発行された。

このドキュメントは、International Software Testing Qualifications BoardのArmin Born, Filipe Carlos, Wim Decoutere, István Forgács, Matthias Hamburg (Product Owner), Attila Kovács, Sandy Liu, François Martin, Stuart Reid, Adam Roman, Jan Sabak, Murian Song, Tanja Tremmel, Marc-Florian Wendland, Tao Xian Fengらによるチームが作成した。

校正をしてくれたJulia Sabatine、テクニカルレビューをしてくれたGary Mogyorodi、常にテクニカルサポートをしてくれたDaniel Pol'an、そして提案や意見をくれたレビューチームと各国委員会に感謝をしたい。

本シラバスのレビュー、コメント、投票に参加したのは以下の方々である:

Current edition (v4.0): Gergely Ágnecz, Laura Albert, Dani Almog, Somying Atiporntham, Andre Baumann, Lars Bjørstrup, Ralf Bongard, Earl Burba, Filipe Carlos, Renzo Cerquozzi, Kanitta Chantaramanon, Alessandro Collino, Nicola De Rosa, Aneta Derkova, Dingguofu, Ole Chr. Hansen, Zheng Dandan, Karol Frühauf, Dinu Gamage, Chen Geng, Sabine Gschwandtner, Ole Chr. Hansen, Zsolt Hargitai, Tharushi Hettiarachchi, Ágota Horváth, Arnika Hryszko, Caroline Julien, Beata Karpinska, Ramit Manohar Kaul, Mattijs Kemmink, László Kvintovics, Thomas Letzkus, Marek Majerník, Donald Marcotte, Dénes Medzihradszky, Imre Mészáros, Krisztián Miskó, Gary Mogyorodi, Ebbe Munk, Maysinee Nakmanee, Ingvar Nordström, Tal Pe'er, Kunlanan Peetijade, Sahani Pinidiya, Lukáš Piška, Nishan Portoyan, Meile Posthuma, Yasassri Ratnayake, Randall Rice, Adam Roman, Marc Rutz, Jan Sabak, Vimukthi Saranga, Sumuduni Sasikala, Gil Shekel, Radoslaw Smilgin, Péter Sótér , Helder Sousa, Richard Taylor, Benjamin Timmermans, Giancarlo Tomasig, Robert Treffny, Shun Tsunoda, Stephanie Ulrich, François Vaillancourt, Linda Vreeswijk, Carsten Weise, Marc-Florian Wendland, Paul Weymouth, Elżbieta Wiśniewska, John Young, Claude Zhang.

Edition 2021-2022 (v3.1): Gery Ágnecz, Armin Born, Chenyifan, Klaudia Dussa-Zieger, Chen Geng (Kevin), Istvan Gercsák, Richard Green, Ole Chr. Hansen, Zsolt Hargitai, Andreas Hetz, Tobias Horn, Joan Killeen, Attila Kovacs, Rik Marselis, Marton Matyas, Blair Mo, Gary Mogyorodi, Ingvar Nordström, Tal Pe'er, Palma Polyak, Nishan Portoyan, Meile Posthuma, Stuart Reid, Murian Song, Péter Sótér, Lucjan Stapp, Benjamin Timmermans, Chris van Bael, Stephanie van Dijck, Paul Weymouth.

その後、TAl Pe'er、Stuart Reid、Marc-Florian Wendland、Matthias Hamburgが、形式的、文法的、表現的な改善を提案し、正誤表3.1.1および3.1.2として発行された。

Edition 2019 (v3.0): Laura Albert, Markus Beck, Henriett Braunné Bokor, Francisca Cano Ortiz, Guo Chaonian, Wim Decoutere, Milena Donato, Klaudia Dussa-Zieger, Melinda Eckrich-Brajer, Péter Földházi Jr, David Frei, Chen Geng, Matthias Hamburg, Zsolt Hargitai, Zhai Hongbao, Tobias Horn, Ágota Horváth, Beata Karpinska, Attila Kovács, József Kreisz, Dietrich Leimsner, Ren Liang, Claire Lohr, Ramit Manohar Kaul, Rik Marselis, Marton Matyas, Don Mills, Blair Mo, Gary Mogyorodi, Ingvar Nordström, Tal Peer, Pálma Polyák, Meile Posthuma, Lloyd Roden, Adam Roman, Abhishek Sharma, Péter Sótér, Lucjan Stapp, Andrea Szabó, Jan te Kock, Benjamin Timmermans, Chris Van Bael, Erik van Veenendaal, Jan Versmissen, Carsten Weise, Robert Werkhoven, Paul Weymouth.

Edition 2012 (v2.0): Graham Bath, Arne Becher, Rex Black, Piet de Roo, Frans Dijkman, Mats Grindal, Kobi Halperin, Bernard Homès, Maria Jönsson, Junfei Ma, Eli Margolin, Rik Marselis, Don Mills, Gary Mogyorodi, Stefan Mohacsi, Reto Mueller, Thomas Mueller, Ingvar Nordstrom, Tal Pe'er, Raluca MadalinaPopescu, Stuart Reid, Jan Sabak, Hans Schaefer, Marco Sogliani, Yaron Tsubery, Hans Weiberg, Paul Weymouth, Chris van Bael, Jurian van der Laar, Stephanie van Dijk, Erik van Veenendaal, Wenqiang Zheng, Debi Zylbermann.

©International Software Testing Qualifications Board

0 イントロダクション

0.1 本シラバスの目的

本シラバスは、テストアナリスト向けの国際ソフトウェアテスト資格Advanced Levelのベースとなる。ISTQB® は、本シラバスを以下の主旨で提供する:

1 メンバー委員会に対し、各国語への翻訳および教育機関の認定の目的で提供する。メンバー委員会は、本シラバスを各言語の必要性に合わせて調整し、現地の出版物に合わせて参考文献を追加することができる。
2 認定委員会に対し、本シラバスの学習の目的に合わせ、各国語で試験問題を作成する目的で提供する。
3 教育機関に対し、コースウェアを作成し、適切な教育方法を決定する目的で提供する。
4 受験志願者に対し、認定試験準備の目的で提供する(研修コースの一部として行う場合と、独学で行う場合のいずれも含む)。
5 国際的なソフトウェアおよびシステムエンジニアリングのコミュニティに対して、ソフトウェアおよびシステムのテストに対する専門性を向上させ、書籍や記事のベースとして提供する。

0.2 テスト技術者資格制度Advanced Level Test Analyst

ISTQB® Advanced Level Test Analyst (CTAL-TA) 認定は、ソフトウェア開発ライフサイクル全体にわたって、構造化された徹底的なソフトウェアテストを実施するために必要なスキルを提供する。標準的なテストプロセスの各ステップにおけるテストアナリストの役割と責任を詳述し、重要なテスト技法について解説する。Advanced Level Test Analyst認定は、Foundation Levelの認定資格を持ち、テスト分析とテスト技法に関する専門知識をさらに深めたい技術者を対象としている。

本シラバスでは、テストアナリストとは、次のような役割として位置づける:

- テストの技術的側面よりも、顧客のビジネスニーズに重点を置く。
- 主に機能テストを行うが、使用性、適応性、インストール性、相互運用性テストなど、ユーザーに焦点を当てた非機能テストにも貢献する。
- ホワイトボックステスト技法よりも、ブラックボックステスト技法および経験ベースのテスト技法を用いる。
- 欠陥予防技法を用いて、テストの有効性を向上させる。

0.3 テスト担当者のキャリアパス

ISTQB® スキームは、キャリアのあらゆる段階にあるテストプロフェッショナルを支援する。ISTQB® Advanced Level Test Analyst認定資格取得者は、他のコアAdvanced Level（例：テストマネジメント、テクニカルテストアナリスト）や、その後 ISTQB® Expert Level（例：テストマネジメント、テストプロセス改善）へと興味を広げられる。アジャイルソフトウェア開発におけるテストのスキルを身に付けたい人は、Agile Technical TesterまたはAgile Test Leadership at Scaleの認定を検討することができる。さらに、スペシャリスト系のストリームでは、特定のテスト技術やアプローチ、特定の品質特性やテストレベル、あるいは特定

©International Software Testing Qualifications Board

の業界領域におけるテストに焦点を当てた認定制度を提供している。ISTQB認定テスト担当者制度に関する最新情報は、www.istqb.org を参照のこと。

©International Software Testing Qualifications Board

0.4 ビジネス成果

この節では、Advanced Level Test Analyst 認定取得者に期待されるビジネス成果を示す。

Advanced Level Test Analystの認定テスト担当者は、次のビジネス成果を達成できる。

	説明
TA-BO1	ソフトウェア開発ライフサイクルに基づいた適切なテストを支援し、実施する。
TA-BO2	リスクベースドテストの原則を適用する。
TA-BO3	テスト目的の達成を支援するために、適切なテスト技法を選択し、適用する。
TA-BO4	適切な詳細度と品質の文書を提供する。
TA-BO5	実施すべき機能テストの適切な種類を決定する。
TA-BO6	非機能テストに貢献する。
TA-BO7	欠陥予防に貢献する。
TA-BO8	ツールの活用により、テストプロセスの効率を改善する。
TA-BO9	テスト環境とテストデータの要件を明確にする。

0.5 学習の目的と知識レベル

学習の目的はビジネス成果を支援し、テスト技術者資格制度Advanced Level Test Analyst試験問題作成に使用する。

一般的に、本シラバスの内容は、「イントロダクション」と「付録」を除き、すべて出題範囲である。試験問題は、K1レベル(下記参照)のキーワードの知識、またはそれぞれの知識レベルの学習の目的を確認する。

具体的な学習の目的とその知識レベルは各章の冒頭に示され、以下のように分類されている:

- K2: 理解
- K3: 適用
- K4: 分析

各章の見出しの下にキーワードとして記載されているすべての用語について、ISTQB®用語集に記載されている正しい名称と定義を、たとえ学習の目的に明示していなくても、覚えておかなければならない(K1)。

学習の目的の詳細と例は*7章*に記載している。

0.6 Advanced Level Test Analystの資格認定試験

Advanced Level Test Analyst認定試験は、本シラバスに基づく。試験問題の解答には、本シラバスの複数の章に基づく資料の使用が必要な場合がある。シラバスの「イントロダクション」と「付録」を除くすべての章が出題範囲である。標準や書籍は参考文献として含まれているが、それらに関して本シラバス自体で要約されている以上の内容は試験対象外である。

©International Software Testing Qualifications Board

詳細については、Advanced Level Test Analyst v4.0と互換性のあるExam Structures and Rulesを参照のこと。

ISTQB® 認定Advanced Level Test Analystの受験資格は、ソフトウェアテストに関心を持っていることである。しかし、受験者は次の事項も強く推奨される:

- システムテスト担当者またはユーザー受け入れテスト担当者として6ヶ月間、またはソフトウェア開発担当者として6ヶ月間の経験など、ソフトウェア開発またはソフトウェアテストの少なくとも最低限の経験があること。
- ISTQB® の基準で認定(ISTQB公認メンバー委員会のいずれかによる)されたコースを受講すること。

受験要件: ISTQB® Foundation Level資格は、Advanced Level Test Analyst認定試験を受験する前に取得すること。

0.7 認定審査

ISTQB® メンバー委員会にて、本シラバスに従ったコースを提供する教育機関を認定する。教育機関は、メンバー委員会または認定を行う機関から認定ガイドラインを入手すること。認定されたコースは、本シラバスに準拠していると認められ、コースに ISTQB® 試験を含めることができる。本シラバスの認定ガイドラインは、ISTQB® Processes Management and Compliance Working Group が発行する一般認定ガイドラインに従う。

0.8 標準の取り扱い

IEEEやISOといった国際標準化組織は、品質特性やソフトウェアテストに関連する標準を発行している。本シラバスでは、これら標準を参照する。これらの参考文献の目的は、(品質特性に関する ISO/IEC 25010 の参考文献のように)枠組みを提供すること、あるいは読者が希望する場合に追加情報の情報源を提供することである。ISTQB®シラバスは、標準を参考文献として使用していることに留意すること。標準は試験を目的とするものではない。標準の詳細については、第 6 章「参考文献」を参照のこと。

0.9 詳細レベル

本シラバスの詳細なレベルは、国際的に一貫した教育と試験を可能にする。このゴールを達成するために、シラバスは以下のように構成されている:

- Advanced Level Test Analystの意図を説明する一般的な指導対象
- 想起しなければならない用語のリスト
- 達成すべき認知的な学習の成果を説明した、知識領域ごとの学習の目的5
- 一般に認められた文献や標準などの情報源の参照を含む主要なコンセプトの説明

本シラバスの内容は、ソフトウェアテストの知識領域全体を説明していない。詳細レベルは、Advanced Level Test Analystの教育コースでカバーされることを示している。

©International Software Testing Qualifications Board

本シラバスでは、ISTQB® Glossary（ISTQB-Glossary, 2024）に従ったソフトウェアテストおよび品質保証で使う用語（すなわち、用語の名称および意味）を用いる。関連分野の用語については、それぞれの用語集を参照されたい。要求エンジニアリングについては IREB-CPRE（IREB-Glossary, 2024）、ソフトウェアエンジニアリングについては IEEE-Pascal（Computer Society et al., 2024）である。

0.10 本シラバスの構成

試験可能な内容の章は5つある。各章の一番上の見出しには、認定研修コースがその章の内容をカバーするために最低限必要な学習時間が明記されている。認定研修コースの場合、本シラバスは、最低**20.25時間（1215分）**の講義を必要とし、5つの章で以下のように配分する：

- 第1章（225分）テストプロセスにおけるテストアナリストのタスク
  - 受講者は、テストアナリストがさまざまなソフトウェア開発ライフサイクルにどのように関わるかを学ぶ。
  - 受講者は、テストアナリストがさまざまなテスト活動にどのように関わるかを学ぶ。
  - 受講者は、作業成果物に対してテストアナリストが行うタスクについて学ぶ。
- 第2章（90分）リスクベースドテストにおけるテストアナリストのタスク
  - 受講者は、テストアナリストがプロダクトリスク分析にどのように貢献するかを学ぶ。
  - 受講者は、リグレッションテストの範囲を決定するために、変更の影響を分析する方法を学ぶ。
- 第3章（615分）テスト分析とテスト設計
  - 受講者は、ドメインテスト、組み合わせテスト、ランダムテストなど、データに基づくテスト技法について学ぶ。
  - 受講者は、CRUDテスト、状態遷移テスト、シナリオベースドテストなど、振る舞いに基づくテスト技法について学ぶ。
  - 受講者は、デシジョンテーブルテストやメタモルフィックテストなど、ルールに基づくテスト技法について学ぶ。
  - 受講者は、セッションベースドテストやクラウドテストなど、経験ベースのテストについて学ぶ。
  - 受講者は、プロダクトリスクを軽減するために適切なテスト技法を選択する方法を学ぶ。
- 第4章（60分）品質特性のテスト
  - 受講者は、いくつかの種類の機能テストの実施方法を学ぶ。
  - 受講者は、使用性テスト、柔軟性テスト、互換性テストなどの非機能テストタイプに貢献するために、機能性に関する具体的な知識をどのように活用するかを学ぶ。
- 第5章（225分）ソフトウェア欠陥予防
  - 受講者は、さまざまな欠陥予防の実践例を学ぶ。
  - 受講者は、フェーズ内封じ込めを支援するさまざまなアプローチについて学ぶ。
  - 受講者は、類似欠陥の再発を軽減する方法を学ぶ。

©International Software Testing Qualifications Board

1 テストプロセスにおけるテストアナリストのタスク - 225分

キーワード
ハイレベルテストケース、キーワード、キーワード駆動テスト、ローレベルテストケース、ソフトウェア開発ライフサイクル、テスト分析、テストアナリスト、テストケース、テスト条件、テストデータ、テスト設計、テスト環境、テスト実行、テスト実装、テストオラクル、テストスクリプト、テストウェア

第1章の学習の目的

1.1 ソフトウェア開発ライフサイクルにおけるテスト

TA-1.1.1 (K2) さまざまなソフトウェア開発ライフサイクルにおけるテストアナリストの関わりについて要約する。

1.2 テスト活動への関与

TA-1.2.1 (K2) テスト分析の一環としてテストアナリストが行うタスクを要約する。

TA-1.2.2 (K2) テスト設計の一環としてテストアナリストが行うタスクを要約する。

TA-1.2.3 (K2) テスト実装の一環としてテストアナリストが行うタスクを要約する。

TA-1.2.4 (K2) テスト実行の一環としてテストアナリストが行うタスクを要約する。

1.3 作業成果物に関するタスク

TA-1.3.1 (K2) ハイレベルテストケースとローレベルテストケースを区別する。

TA-1.3.2 (K2) テストケースの品質基準について説明する。

TA-1.3.3 (K2) テスト環境要件の例を挙げる。

TA-1.3.4 (K2) テストオラクル問題とその解決策について説明する。

TA-1.3.5 (K2) テストデータ要件の例を挙げる。

TA-1.3.6 (K3) テストスクリプトを開発するためにキーワード駆動テストを使用する。

TA-1.3.7 (K2) テストウェアをマネジメントするツールの種類を要約する。

©International Software Testing Qualifications Board

テストプロセスにおけるテストアナリストのタスクについて

Foundation Levelシラバス(ISTQB-CTFL, v4.0.1)では、テストにおける 2 つの主要な役割として、テストマネジメントの役割とテストを行う役割の記述がある。本シラバスでは、ソフトウェアのビジネス側面のテストを担当する役割の人をテストアナリスト(TA)と呼ぶ。この責務を担う人が専任のポジションや役割に割り当てられることはまれであるが、TAには明確に定義されたタスクと必要となるコンピテンシーがある。テストレベルに関しては、TAはシステムテスト、受け入れテスト、システム統合テストに重点を置く。品質特性に関しては、TAのコンピテンシーは機能適合性に重点を置き、使用性、適応性、インストール性、相互運用性など、ユーザーが直面する特定の非機能品質特性もカバーする。

1.1 ソフトウェア開発ライフサイクルにおけるテスト

1.1.1 さまざまなソフトウェア開発ライフサイクルにおけるテストアナリストの関与

テスト活動の構成は、採用するソフトウェア開発ライフサイクル(SDLC)によって異なる場合がある。したがって、TAのテスト活動への関与は、採用するSDLCモデルによって異なることがある。

シーケンシャル開発モデルでは、開発活動はフェーズごとに行い、前のフェーズが完了した後に次のフェーズを開始する。通常、これらの活動にはほとんど重複がない。したがって、TAのタスクは、通常、時間の経過とともに変化する。SDLC の初期のフェーズでは、TAはテスト計画の支援に集中する。TAは、テストベースが作成されるときに、テスト分析を開始する。テスト設計とテスト実装は、ソフトウェアの設計と実装と並行して行う。最終的に、TAはテストを実行し、SDLC の後期のフェーズでテスト完了を支援する。

インクリメンタル開発モデルでは、ソフトウェアをより小さく、マネジメントしやすいインクリメントに分割する。各インクリメントは独立して開発およびテストをする。したがって、TAは、各インクリメントに対して同じ活動(すなわち、テスト分析、テスト設計、テスト実装、テスト実行、テストマネジメント支援)を行う。ただし、TAの作業は、インクリメントごとに異なる形で編成される場合がある。テストの焦点は新規または変更したフィーチャーに置く。さらに、リグレッションのリスクが高まるため、TAは、リファクタリングとリグレッションテストスイートの組み立てに特に注意を払わなければならない。

イテレーティブ開発モデルでは、開発プロセスは循環的である。プロジェクトは、プロトタイピング、テスト、洗練、デプロイのサイクルを繰り返す。TAの役割はダイナミックで適応的である。TAは、開発担当者やビジネス担当者と密接に協力し、進化するプロダクトに適応する。TAは、ソフトウェアの進化に合わせてテスト条件やテストケースを調整、および修正し、各イテレーションにおいてテストプロセスを改善するためのフィードバックを提供する。

イテレーションの頻度が高ければ高いほど、TAによるリグレッションテストの継続的なメンテナンスと拡充がより重要になる。

SDLCは、さまざまなモデルや特定の技術やアプローチの要素を組み合わせることができる(例えば、アジャイルソフトウェア開発は、イテレーティブモデルとインクリメンタルモデルの両方の側面を組み合わせている)。そのような場合、TAの関与は、SDLCの特定の特性とそれらがどのように組み合わされているかに依存する。TAがSDLCの初期段階から関与することは、すべてのSDLCモデルに共通するよい実践例である。

1.2 テスト活動への関与

Foundation Levelのシラバス(ISTQB-CTFL, v4.0.1)には、テストプロセスにおける7つの活動が記述されている。

©International Software Testing Qualifications Board

TAは、主にそのうちの 4 つ、テスト分析、テスト設計、テスト実装、テスト実行に焦点を当てている。これら4つの活動におけるTAのタスクは、以下の項で詳述する。

1.2.1 テスト分析

テスト分析では、TAはテストベースの完全性を確認し、テストに関連する追加情報を収集する。これには、文書だけでなく、ユーザーストーリーの共同執筆における会話などの口頭の情報も含まれる(ISTQB-CTFL(v4.0.1)、4.5.1項を参照)。テストベースの変更は、テストマネジメントと連携してテストスコープを調整することにつながることがある。

テスト分析を効果的に進めるために、TAは以下の開始基準を確認する:

- テスト計画が行われており、テスト範囲、テスト目的、テストアプローチが明確になっている。
- テストベース(要件やユーザーストーリーなどの情報を含む)を定義している。
- すでに識別したプロダクトリスクについて、必要に応じて評価および文書化をしている。

TAは、テストベースに含まれる可能性のある欠陥を識別し、テストベースのテスト可能性を評定するため、テストベースを評価し、プロダクトオーナーに早期にフィードバックを提供する。これには、適用するテスト技法に従ってシステム動作をモデル化することも含まれる(*3*章、5.2.1項を 参照)。レビュー技術もプロセスの一部として適用する(*5.2.2*項を参照)。直接修正しない場合、テストベースに関する欠陥は文書化しなければならない。さらに、TAは必要なテストオラクルを決定する(*1.3.4*項を参照)。

TAは、スコープ内の各テストアイテムのテスト条件を定義し、優先順位を付ける。テスト条件は、テスト目的(ISTQB-CTFL v4.0.1、1.1.1項を参照)に対応し、テストベースの要素にトレーサビリティを持たなければならない。テスト条件の範囲と焦点は、プロダクトリスクを考慮する。インクリメンタル開発モデルまたはイテレーティブ開発モデルでは、影響度分析に基づいてリグレッションテストの範囲を決定することを含める。アジャイルソフトウェア開発では、テスト条件をユーザーストーリーのリスクを反映した受け入れ基準として表現することができる。

TAは、「画面Xの機能性」のようなハイレベルのテスト条件から始めて、段階的に進めることができる。次にTAは、「画面Xは、1桁短い口座番号を拒否する」といった、より詳細なテスト条件を定義する。このアプローチは、十分なカバレッジを支援し、テスト設計の早期開始を可能にする、例えば、ユーザーストーリーがまだ洗練されていない場合などである。

TAは、テストベースを明確に理解し、テストすることがテスト目的に沿っていることを確認するために、ステークホルダーを巻き込んでテスト条件をレビューする。

1.2.2 テスト設計

テスト設計では、明示されたテスト目的を達成するために、どのようにテストを実施するかを記述する。これは、通常、テストケースを用いて行われる。テスト設計の実施方法は、求められるカバレッジ、テストベース、SDLC、プロジェクトの制約、および、テスト担当者の知識と経験など、多くの要因に依存する。

テスト設計中、TAは、どのエリアでローレベルテストケースまたはハイレベルテストケースが適切になるかを決定する(*1.3.1*項を参照)。いずれの場合もTAは合格/不合格の基準を明確にしなければならない。TAは、品質基準(*1.3.2*項を参照)に従って、新規または変更したテスト条件のためのテストケースを設計する。リグレッションテストの場合は、既存のハイレベルテストケースの選択、または優先順位付けに基づく既存のローレベルテストケースを調整することで通常は十分である。

TAは、テストベース、テスト条件、テストケースの間のトレーサビリティを確立する。経験ベースのテストでは、テストケースを必ずしも文書化しない。その代わりに、テスト条件(その他のものも含めて)がテスト実行の指針となる。TAは、ハイレベルのテスト目的に基づいて、いくつかのテストケースを設計することができる。これらの作業に加え、TAはテスト環境要件を定義し(*1.3.3項*を参照)、テストデータの要件を識別、作成、明確化する(*1.3.5項*を参照)。

©International Software Testing Qualifications Board

TAは、テスト計画書で定義された終了基準を用いて、十分なテストケースが設計されたかどうかを判断する。しかし、残存リスクレベルやプロジェクト制約（予算や時間など）のような他の終了基準も、テスト設計の終了時期を示す場合がある。

テスト設計はツールによって支援されることもあるが、ツールに縛られないために、特定のツールや技術に依存しない形で行うべきである。テスト設計は、テスト技法を用いた体系的なアプローチを適用するか、あるいはそれ以外の場合にはアドホックに実施する。

テストケースは伝達的な役割を持ち、関連するステークホルダーが理解できるものでなければならない。テストケースは、その作成者によって実行されるとは限らないので、他のテスト担当者は、その実行方法、テスト目的（すなわち、その背後にあるテスト条件）、およびその重要性を理解する必要がある。また、テストケースは、テストを実装または故障発生時に再実行する可能性のある開発担当者や、テストの承認をする監査担当者にも理解可能でなければならない。

1.2.3 テスト実装

テスト実装中、TAはテスト実行に必要なテストウェアを提供する。TAは、テストプロシジャーとテストスクリプトをテストスイートにまとめたり、自動化のためのテストケースを提案したりする。テストプロシジャーを定義するには、テスト実行順序に影響を及ぼす可能性のある制約や依存関係を注意深く識別する必要がある。テストプロシジャーには、一連のテストケースに含まれるステップに加え、初期の事前条件の設定（例：データリポジトリからのテストデータの投入）、期待結果および事後条件の検証、実行後のリセット手順（例：データベース、環境、システムのリセット）を含める。

TAは、リスク分析とテスト計画書作成時に識別した優先順位付け基準に基づいて、テスト実行のためのテストプロシジャーとテストスクリプトに優先順位を付け、テスト対象の現行バージョンで実行すべきテストプロシジャーまたはテストスクリプトを識別する。これにより、（新しいフィーチャー用のテストやリグレッションテストなど）関連するテストを特定のテストランで一緒に実行することができる。TAは、テストベースと、テストプロシジャー、テストスクリプト、テストスイートなどの他のテストウェアとの間のトレーサビリティを更新する。

TAは、テスト実行順序を定義することにより、効率的なテスト実行を可能にするためのリソースの割り当てを含むテスト実行スケジュールの策定において、TMを支援することができる（ISTQB-CTFL, v4.0.1, 5.1.5項参照）。

TAは、データベースやその他のリポジトリ（*1.3.5* 項を参照）に投入するための入力データや環境データを作成する。このデータは、特定のテスト目的を支援するために「意義に沿った」ものでなければならない。

TAはまた、テスト環境が完全にセットアップされ、テスト実行の準備が整っていることを検証すべきである（*1.3.3*項参照）。この検証は、スモークテストを設計し、実行することによって実施するのが最適である。テスト環境は、テスト実行を通じてテスト対象内の欠陥を明らかにし、故障が発生しない場合には正常に動作し、必要に応じて本番環境またはエンドユーザー環境を適切に再現すべきである。

テスト実装中に実施する作業の詳細度やそれに伴う複雑さは、テスト条件やテストケースの詳細度に影響を受ける場合がある。場合によっては、規制上のルールが適用され、テストウェアが該当する標準（例えば、*RTCA DO-178C*, 2011）への適合を示す証拠を提供すべきことがある。

1.2.4 テスト実行

テスト実行は、テスト実行スケジュールに従って行われる。TAが行う典型的な作業は、テストの実行、実際の結果と期待結果の比較、不正の解析、欠陥の報告、テスト結果の記録である。TAはテストを手動で実行する。これには探索的テスト、テストプロシジャーの実行、リグレッションテスト、確認テストが含まれる。探索的テストでは、TAはテストチャーター用いてセッションベースドテストを行うことができる（*3.4.1*項を参照）。TAは自動テストスクリプトを実行することもできるが、故障の場合に自動テストスクリプトを実行して分析を行うのは、開発担当者、テスト自動化エンジニア（TAE）、またはテクニカルテストアナリスト（TTA）の仕事の場合がある。

©International Software Testing Qualifications Board

TAは、手動または自動テスト実行で発覚した不正を分析し、その推定原因を特定する。不正はテスト対象の欠陥に起因する場合もあるが、他にも、事前条件の欠如、テストデータの誤り、テストスクリプトやテスト環境の欠陥、仕様の誤解などの原因も考えられる。TAは、テスト実行の実際の結果を記録し、観測された故障に基づいて欠陥を伝達し、必要に応じて報告を行う。

TAは、テスト結果を考察することにより、テストベースと他のテストウェア間のトレーサビリティを更新する。この情報により、テスト結果を高レベルのリスクまたはカバレッジ情報に変換することができ、ステークホルダーが情報に基づいた判定を行うことができる。あるテスト条件に関連するテストケースがいくつ合格したのか、あるいは不合格なのかを関係者に明らかにする。

これらの典型的なタスクに加え、TAは以下のタスクを含むテスト結果を評価する:

- 欠陥クラスターを認識することで、テスト対象の特定の部分について、より多くのテストが必要であることを示すことができる(*5.3.1*項参照)。
- 不合格となった自動テストを手動で再実行し、自動化したテストが偽陽性結果を出さなかったことを確認する。
- 前回のテストで学んだことに基づいて、追加のテストを提案する。
- テストで得た情報から新たなリスクを識別する。
- テスト設計やテスト実装(テストプロシジャーの改善など)、あるいはテスト対象システムの改善を提案する。
- リファクタリング、スコープ調整、テスト自動化など、リグレッションテストスイートの改善を提案する(*2.2.1*項を参照)。

1.3 作業成果物に関するタスク

TAは、自身が責任を負う作業成果物の品質を確実にしなければならない。これには、テストケース、テスト環境、テストデータ、テストオラクル、テストスクリプトが含まれる。本節では、テストウェアに関連するTAのタスクと、テストウェアをマネジメントするためのツールのタイプについて説明する。

1.3.1 ハイレベルテストケースとローレベルテストケース

ハイレベルテストケース(抽象テストケースまたは論理テストケースとも呼ばれる)は、どのテスト条件がテストケースでカバーされるかを示すことで、テスト対象が検査される状況を記述する。したがって、ハイレベルテストケースは、関連するすべてのテスト条件をテストがカバーしていることを確認するのに適している。ハイレベルテストケースには、事前条件、入力データ、期待される出力、事後条件などの具体的な情報は含まれない。これらはすべて抽象的なレベルで表現される(例えば、「複数の本を注文し、注文価格が割引になる;期待結果:割引を適用する」)。

ローレベルテストケース(具体的テストケースまたは物理的テストケースとも呼ばれる)は、ハイレベルテストケースを詳細化したものである。ローレベルテストケースでは、どのようなデータを準備する必要があるのか、テスト担当者が(必要であれば)どのような行動を取らなければならないのか、そして具体的にどのような期待結果が得られるのかを記述する。ローレベルテストケースには、特定の事前条件、入力データ、期待結果、および事後条件が含まれる(例:「書籍B1(10ドル)とB2(20ドル)を注文し、合計注文金額は30ドル。期待結果:10%の割引が適用され、合計金額は27ドルとなる」)。

通常、TAは最初にハイレベルテストケースを設計し、それがローレベルテストケース開発のベースとなる。ひとつのハイレベルテストケースは、1つ以上のローレベルテストケースで実装できる。テストケースはハイレベルのままで、具体的な情報はテスト実行中にTAが決定することもある。例えば、TAは、セッションベースドテストのテストチャーター内でテスト目的を作成する際に、ハイレベルテストケースを指針とすることができ、テスト実行中にそれらの目的を具体化できる(3.4.1 項を参照)。

ハイレベルテストケースからローレベルテストケースへの移行は、単に具体的な値を記入するだけではない。概念的

©International Software Testing Qualifications Board

なものから技術的なものへのステップでもある。特に具体的なテストデータが必要な場合、テスト設計からテスト実装へ先送りすることが多い。TAは、ローレベルテストケースの実行に必要なものがすべて明確にわかっていなければならない(Koomen et al., 2006)。実際には、多くのテストケースはハイブリッドで、具体的でありながら、ある面では抽象的である。これは多くの場合、テストケースの保守性と理解性のトレードオフによるものである。

1.3.2 テストケースの品質基準

テストケースの品質をおろそかにすると、保守コストの増大、理解性の低下、テスト実行の遅延など、多くの問題を引き起こす可能性がある。テストケースの品質基準は、より保守性の高いテストケースへの第一歩である。これには以下がある:

- **正確性** テストケースは、ベースとなるテスト条件の正確な検証を容易にしなければならない。
- **実現可能性** テストケースの実行が可能でなければならない。
- **必要性** すべてのテストケースは、そのタイトルや要約で表現されているように、明確なテスト目的をカバーすべきである。重複は避けるべきである。テストすべきでないものにはテストケースを設計すべきではない。
- **理解容易性** テストケースは、作成者以外の人がレビュー、修正、実行する可能性がある。TAは、当たり前のことを説明することなく、関係者全員が理解できる言語と形式でテストケースを書くべきである。複雑度の高いテストケースは単純化するか分割する。
- **トレーサビリティ** テストケースは、TAが常に最新の状態に保つことができるように、テスト条件、要件、リスクにトレーサビリティを持たなければならない(ISTQB-CTFL v4.0.1、1.4.4 項を参照)。
- **一貫性** 言語、書式、構造の一貫性は、テストケースの理解と保守を容易にする。TAは、この目的のために用語集を使用することができる。
- **明確性** 偽陰性および偽陽性結果を避けるため、テストケースの解釈はひとつだけであるべきである。「適切な」、「必要に応じて」、「いくつかの」といった曖昧な表現は避けるべきである。
- **完全性** 必要となるテストデータ(*1.3.5項*を参照)および実際の結果との比較時に判断に迷いが生じない明確な期待結果を含む、必要な属性(例えば、*ISO/IEC/IEEE 29119-3*, 2021 参照)がすべて存在すべきである。
- **簡潔性** テストケースの粒度(すなわち、多くのテストアクションを持つ1つの大きなテストケースにするか、複数の小さなテストケースにするか)は、テストベースとテスト条件に整合しているべきである。少数のカバレッジアイテムに焦点を絞った小さなテストケースが望ましい。故障の原因の特定が容易で、テストプロシジャーやテストスイートに柔軟に組み合わせることができ、テスト実行中に故障が発生してもそれ以降のテストがブロックされないためである。

テストケースの形式と詳細レベルは、プロジェクトとプロダクトの状況に依存し、テストチーム内で合意されなければならない

1.3.3 テスト環境要件

テスト環境は、手動と自動の両方のテスト実行にとって、重要な成功要因である。テスト環境の実装は、テスト可能性、欠陥検出、全体的なテストコスト、および、テスト結果の信頼性に影響する。テスト環境で合格または不合格となったテストケースは、本番環境で実行しても同じテスト結果になるようにすべきである。テスト環境は、堅牢で、予測可能で、必要であればテスト自動化フレームワークと統合されていることが理想的である。TAは、以下の分析に基づいて、テスト設計中にテスト環境の要件を定義することができる:

- テスト環境要件は、テスト条件、テストケース、テストデータ要件などと関連しており、テスト実行の事前条件が満たされるようにテスト環境を設定し、維持するために必要な条件を記述する。
- テストレベルおよびテストタイプは、テスト環境の柔軟性の度合いと、本番環境との類似の度合いの間のトレードオフに影響する。

©International Software Testing Qualifications Board

- コンポーネントやシステムの可用性および独立性は、テストダブル(スタブやドライバーなど)を使用する必要があるかを示す場合がある。

テスト環境要件は、テスト環境の構成要素を記述するものであり、これらはハードウェア、ミドルウェア、ソフトウェア、仮想化サービス、ネットワーク、インターフェース、ツール、セキュリティ、構成、および場所など、いくつかの種類に分類できる。各テスト環境の構成要素について、要件には以下の情報を含めるべきである(ISO/IEC/IEEE 29119-3, 2021):

- 一意の識別子 - トレーサビリティの目的で使用される。
- 説明 - 要件に応じて実装できるよう十分な詳細さで記述する。
- 責任 - 誰がそれを利用可能にする責任があるかを明示する。
- 必要期間 - その構成要素がいつからいつまで必要かを識別する。
- 忠実度 – 構成要素が本番環境をどの程度再現しているか、またはどの程度逸脱しているかの度合い。

この要件は、セットアップ、バックアップとリストア、セキュリティ要件、テスト環境の変更能力、ロールと認可など、テスト環境の包括的なニーズにも対応する必要がある(Koomen et al.,2006)

TAは、テスト環境要件を明確かつ簡潔で、首尾一貫した形で文書化すべきである。TAは、図や表を使用することができる。冗長性や不必要な文書化を避けるために、既存のテスト環境を参照し、テストレベル固有のニーズに焦点を絞ることができる。関連するステークホルダー(開発担当者、TTA、テスト自動化エンジニア、ビジネスアナリスト、スポンサー、プロダクトオーナーなど)も、テスト環境要件のレビュー、承認、更新を行うべきである。

1.3.4 テストオラクルの決定

テストオラクルは、動的テストにおいて期待結果を決定するために必要となる。理想的には、テストベースがテストオラクルを提供する(例えば、テキストに書かれた、または形式的な仕様の中)。テスト対象に関する人間の経験や知識も、テストオラクルとして機能することができる。コスト効率の観点から、自動化されたテストオラクルが必要となる場合もある(例:入力が自動生成される場合や、人間のオラクルが高コストである場合)。

テストベースの品質や完成度、またはシステムの特性によっては、コスト効率の良いテストオラクルが利用できない場合がある。これは*テストオラクル問題*として知られている。テストオラクル問題の典型的な要因としては、データ関連の複雑性、非決定性(AIベースシステムなど)、確率的な振る舞い、ならびに要件の欠落や曖昧さなどがある。

テストオラクル問題に対する既知の解決策には、以下のようなものがある(Barr et al., 2014):

- 疑似オラクルとは、テスト対象と同じ仕様を満たす、独自に開発されたシステムのことである(例:レガシーシステム、テスト対象の簡易版)。複雑なテスト対象に対して専用の疑似オラクルを開発することは、高価になる可能性があるが、クリティカルなシステムにおいては一般的である。
- モデルベースドテストでは、テストモデルの一部としてテストオラクルを形式化することができる。これにより、期待される出力の生成と、モデルからのテストの導出が可能になる。振る舞いに基づくテスト技法は、自動テストオラクルの実装を伴うことが多い(*3.2.2* および*3.2.3* 項を参照)。
- プロパティベースのテストでは、テスト対象の定義したプロパティを使用して、入力と個々のテストケースの期待結果との関係を検証する。そのような関係が満たされない場合、テストケースは不合格となる。このソリューションはテスト自動化に適しているが、その有効性は関係に依存し、その関係を判断するのは難しいことがある。
- メタモルフィックテスト(*3.3.2*項を参照)。
- 人間オラクルは、期待結果を判断する人間の能力を利用する。人的資源はコストがかかり、不足しがちである。しかし、一部のテストアプローチ(探索的テストなど)では、人間オラクルが好まれる。

アサーションは、自動テストオラクルを実装するために、テスト自動化コードまたはテスト対象自体に組み込むこと

©International Software Testing Qualifications Board

ができる。アサーションは、テスト対象の状態や振る舞いを検証する実行ステートメントである。テスト対象に組み込む場合は、通常、タスクの継続に必要なものだけを検証する。

1.3.5 テストデータ要件

テスト設計中、TAは、テスト実行のために準備または払い出しが必要なテストデータを識別し、用意を依頼する。検討事項には、目的、形式、利用時のコンテキストが含まれる。(*ISO/IEC/IEEE 29119-3*, 2021, 8.5節)にもテストデータの要件が記載されている。重要な点は以下のとおりである:

- **本番データとの類似性** 本番データは実世界のデータを反映するが、ばらつきに乏しい可能性がある。合成データはばらつきをコントロールできる。合成データは、本番データのパターン、分布、外れ値に関する主要な特徴を反映すべきであるが、実世界のデータでのみ発生する特定の欠陥を見落とす可能性がある。本番データがないシステムでは、合成データは現実的なビジネスおよび技術シナリオを反映しなければならない。ペルソナは、多様なユーザーシナリオと行動を反映したデータ作成の指針となる、現実的でユーザー中心のプロファイルを提供することで役立つ。
- **機密性** 機密性の高いテストデータ(例:個人情報)は保護が必要となる。仮名化データは、個人情報を人工的な識別子で置換したものである。匿名化データは、個人を識別できる情報を削除する。必要に応じて、TAは欧州連合のGDPR(Commission, 2016)や米国のHIPAA(Health et al., 2024)のようなデータ保護規制を遵守しなければならない。
- **目的** テストデータは、システムの状態やその構成(システムの日時など)に影響を与える事前条件や期待結果を決定するために極めて重要である。また、ユーザー権限の指定や、データ間の関係の確立(例えば、商品、部門、カテゴリー間の関係)確立も含まれる。
- **カバレッジ基準** テストデータは、選択したテスト技法のカバレッジ基準に沿ったものでなければならない。妥当性確認のテストデータに加え、ネガティブなテストのような無効なテストデータが必要となる場合もある。
- **データ形式** 体系的なデータマネジメント(APIテストなど)では、構造化データ(CSV、JSON、XML、データベースなど)が要件となる場合がある。
- **トレーサビリティ** トレーサビリティは、テストケースに変更がある場合のテストデータの保守性を確保する。
- **保守性** 欠陥検出とメンテナンスを容易にするために、ローレベルテストケースにおけるハードコードされたテストデータは避けるべきである。テストケースは、テストロジックとテストデータを分離すべきである(*1.3.2* 項を参照)。
- **依存関係** 従属データを作成するには一連のステップが必要である。
- **可用性** サービス仮想化は、存在しないサービスやアクセスできないサービスをシミュレーターで再現し、外部のシステムやサービスと相互作用させることで、データの欠落に対処することができる。
- **時間感度とデータの経年変化** 古いデータや時間の影響を受けやすいデータを含むテストケースは、システムの動作に予期しない形または不正確な形で影響を与える可能性がある。

1.3.6 キーワード駆動テストを使用したテストスクリプトの開発

キーワード駆動テストを使用する場合、TAはキーワードを用いてテストスクリプトを作成する。テストスクリプトの実装は、TTA、TAE、または開発担当者の仕事である。

TAは、テストベースの分析や関係者との協働によってキーワードを識別し、定義する。キーワードは、アクションと検証の2つに分類できる。アクションキーワードは、テスト対象(例えば、機能の実行、データの送信、テスト対象内の移動など)。テスト環境(コンフィギュレーションの設定やシミュレーターの起動など)、または他のコンポーネントやシステム(テスト対象のインターフェースのトリガーなど)と相互作用しなければならない。検証キーワードは、テスト対象が実際に生成した結果が期待と一致するかを評価するためのアサーションを表す。

©International Software Testing Qualifications Board

キーワードは、ドメインレイヤーとテストインターフェースレイヤーという、少なくとも2つの抽象化レイヤーに存在する。ドメイン層のキーワードは、ビジネス関連のアクションに対応し、アプリケーションドメインの用語を反映する。これらのキーワードは、テスト対象のインターフェースの技術的な詳細を抽象化する。テストインターフェース層のキーワードは、テストインターフェースを介して、テスト対象、テスト環境アイテム、または他のコンポーネントと通信をする。これらは、最も低い抽象化レイヤーに存在する。付加的な中間レイヤーは、キーワードの保守性を支援する。

キーワードはアトミックであることも、他のキーワードから構成されることもある。キーワードの構造と抽象化レイヤーは独立した属性である。しかし、複合キーワードはより高い抽象レベルに存在することが多く、アトミックキーワードはテストインターフェースレイヤーに存在する傾向がある。

キーワード駆動テストにおけるTAのタスクには、以下のようなものがある:

- キーワードとそのパラメーターの指定
- キーワードテストケースの指定(すなわち、キーワードを使ったテストスクリプトの作成)
- 事前条件、検証アクション、テスト後のテスト環境のクリーンアップなどのキーワードを用いたテストスクリプトへの追加手順の指定
- テスト対象の変更を反映するためのキーワードテストケースの保守
- キーワードテストスクリプトの実行(自動または手動)
- 不合格となったキーワードテストケースの分析(故障原因の特定)

テストベースを分析する際、TAはテスト対象とその環境(ユーザー、他のシステム、デバイスなど)との相互作用を調べる。例えば、ユーザーストーリー「メンバーとして、施設にアクセスできるように自分自身を認証したい」の受け入れ基準が「有効なメンバーカードが認証に使用できる」であるとする。TAは、(アクション)キーワード「メンバー認証」をパラメーター「メンバーカード」と検証キーワード「有効アクセス」で指定することができる。TAは、識別されたキーワードが適切な抽象化レイヤーに存在するかどうかをチェックする。

TAは、必ず以下を満たすようにキーワードを指定しなければならない:

- 動詞(+名詞)を含むこと
- 動詞(+名詞)の命令形を使うこと
- 意味が一意であること
- 適切に文書化されていること
- アプリケーションドメインの語彙を反映していること
- 再利用可能であること

キーワードテストケースを作成するとき、TAはいくつかの欠けているキーワードを認識し、即座に定義することがある。キーワードテストケースは、リストや表など、さまざまな形式で記述できる。

キーワードはプロジェクトのライフサイクルを通じて変化する可能性があり、冗長に定義されやすい(Rwemalika et al.)。本項で述べる推奨事項は、冗長性を回避し、メンテナンスの労力を削減することを目的としている。

キーワード駆動テストはテスト自動化のアプローチであるが、手動テストでもその恩恵を受けることができる。手動テストに適用すれば、後の手動テストから自動テストへの移行を効果的に支援する。

テスト実行の自動化とキーワード駆動テストに関する詳細は、(ISTQB-TTAv4.0)、(ISTQB-TAE, v2.0)、(ISO/IEC/IEEE 29119-5, 2016)に記載がある。

1.3.7 テストウェアのマネジメントに適用するツール

ツールを使用した適切なテストウェアマネジメントは、TAがテストプロセス全体を支援するのに役立つ。ツールは作

©International Software Testing Qualifications Board

業成果物のステータスを提供し、テストモニタリングとテストコントロールを支援できる。テスト対象システムに故障が発生した場合、TAが過去のテストランからテスト結果を確認し、欠陥が発生した時点を分析できる。

テストウェアをマネジメントする上で重要なツールには、以下のようなものがある:

- **テストマネジメントツール** すべての関連テストウェア(テスト条件、テストケース、テストスクリプト、テストスイート、テストランなど)のリポジトリを提供する。このツールは、トレーサビリティ(トレーサビリティマトリクスなどによる)、テストケースの検索、テストランのスケジュール、テスト結果の記録、テスト進捗と品質の全体的なレポートを容易にする。
- **欠陥マネジメントツール** 欠陥解決のプロセスを記録、優先順位付け、モニタリングする。
- **テストデータマネジメントツール** 機密データの保護を含め、テストデータを作成し、保守する(*1.3.5*項を参照)。
- **構成管理ツール** テスト環境の構成や可用性のマネジメントなど、開発、リリース、運用プロセスにおけるテスト関連の活動を促進する。
- **要件マネジメントツール** ハイレベルの要件を格納し、追跡し、これらが SDLC 全体を通じて明確に定義され、バージョンが付けられ、追跡されることを確かにする。

TAは、以下の方法でテストウェアのマネジメントを支援する:

- プロジェクトとリリースを分析し、テストウェアの適切なサブセットを選択する(例:SUTのバージョンに合わせて、バージョンで識別される仕様を選ぶ)。
- テストマネジメントツールにてテストケースを整理するために、適切な機能的構造(機能別またはモジュール別など)または技術的構造(テストタイプ別または環境別など)を定義する。
- テストケースにメタデータを追加する(テスト実行に関わる工数情報や、必要となる特定のテスト環境の情報)。
- 要件、テスト条件、テストケース、テストラン、欠陥間のトレーサビリティを確保する。
- リグレッションテストのための正しいテストスイートを選択する(手動/自動テスト実行用)。
- 古くなったテストケースの識別を含め、テストケースの構成管理を行う。

ツールは、テストプロセスを整理し、追跡し、その品質を確保するのに役立つ。TAは、効率的なテストとテストコントロールのために、テストケースの選択、構造化、保守、トレーサビリティの確保、テストケースのバージョンコントロールを行い、この取り組みを支援する。

©International Software Testing Qualifications Board

2 リスクベースドテストにおけるテストアナリストのタスク - 90分

キーワード

影響度分析、プロダクトリスク、リグレッションテスト、リスク分析、リスクアセスメント、リスクコントロール、リスク識別、リスク軽減、リスクモニタリング、リスクベースドテスト

第2章の学習の目的

2.1 リスク分析
TA-2.1.1　(K2) プロダクトリスク分析におけるテストアナリストの貢献について要約する。

2.2 リスクコントロール
TA-2.2.1　(K4) リグレッションテストの範囲を決定するために変更の影響を分析する。

©International Software Testing Qualifications Board

リスクベースドテストにおけるテストアナリストのタスクについて

リスクベースドテストとは、テストアイテムのリスクレベルに基づいてテストの取り組みに優先順位付けをするテストアプローチである。このアプローチはテストマネージャーが方針を定める。TAは、その実装において積極的な役割を果たす。リスクベースドテストについては、(ISTQB-TM、v3.0)でさらに詳しく解説されている。

リスクベースドテストには、リスク分析とリスクコントロールが含まれる。(ISTQB-CTFL, v4.0.1, 5.2節)

さらに、リスクとそのレベルは静的ではなく時間の経過とともに変化するため、リスクベースドテストには定期的なリスクモニタリングが含まれる。イテレーティブ型ライフサイクルでは、これはチームが決定した頻度(おそらくイテレーションごとに1回)で実行される。その他のライフサイクルでは、その頻度はプロダクトリスクマネジメントの責任者、多くの場合テストマネージャーが設定する。TAは、行われた変更に基づいてリスクレジスターを更新し、前述のリスク軽減アクションを調整することで貢献する。

2.1 リスク分析

リスク分析には、リスク識別とリスクアセスメントが含まれる。本シラバスは、リスクベースドテストが正しく実装されることを確かにするため、TAがリスク分析活動にどのように参加すべきかに焦点を当てる。

TAは、そのシステムに関する独自の知識に加え、通常どのような問題が発生しやすいか、その問題がどのような影響を及ぼすか、そしてテストによってどのようにリスクを軽減できるかについての経験的、かつ直感的な知識を持っていることが多い。このため、プロダクトリスク分析にとってTAは貴重なステークホルダーである。

リスク識別において、TAは、自らの経験と知識を持って、ふりかえり、リスクワークショップ、ブレーンストーミング、チェックリストの作成に貢献する。また、TAは、ステークホルダーへのインタビューを実施し、ステークホルダーの視点から見た最も重大なリスクについて理解を深めることも可能である。

リスクアセスメントの際、TAは他のステークホルダーとともに、以下のようないくつかの要素を見積もることで、リスクレベルの決定に貢献する:

- 影響を受ける機能の使用頻度と深刻度合い
- 影響を受けるビジネス目的の深刻度合い
- 金銭的、環境的、評判上の被害
- テストベースの品質
- 法的または安全上の必要

TAはまた、例えば ISO/IEC 25010(2023)の製品品質モデルを用いて、影響のある品質特性によりプロダクトリスクを分類することも支援する。リスクレベルは、テスト対象に一様に分布していないことが多い。このようなケースでは、TAはテスト対象をテストアイテム(コンポーネント、インターフェース、フィーチャーなど)に分解し、テストアイテムごとに対象となるリスクを個別に評価する。

最後に、プロダクトリスクアセスメントの一環として、TAは、識別した各プロダクトリスクを軽減するための適切なテスト活動を提案する。これらの活動は、静的テストおよび動的テストを含む。リスクレベル、関連するテストアイテムのタイプ、影響を受ける品質特性などの要因に応じて、TAは求められるテストレベル、テストタイプ、テスト技法、テストの独立性レベル、テストの徹底度を示す。シフトレフトの精神に基づき、TAは、テスト工数を最小化するためどのテスト活動が最も早くリスクを軽減できるかを示す。

©International Software Testing Qualifications Board

2.2 リスクコントロール

リスクコントロールには、リスク軽減とリスクモニタリングが含まれる。TAは、システムの機能性とそれらに関連する潜在的なリスクを理解している。したがって、TAの役割は、(ISTQB-CTFL、v4.0.1、5.2.4項)で言及するいくつかのリスク軽減活動において重要である。

- レビューの実施については、本シラバスの*5.2.2*項で説明する。
- 適切なテスト技法とカバレッジレベルの適用については、3.5節で説明する。
- 適切なテストタイプの適用については第4章で説明する。
- リグレッションテストの実行については後述する。

リグレッションテストの主な目的は、変更が加えられた後のテスト対象の品質に対する信頼を確保することである。しかしながら、制約(例えば、時間、予算、テスト環境、テストデータの制約)のために、リグレッションテストを行う中ですべてのリグレッションテストケースを実行することが不可能な場合がある。これは主に手動で実行するテストの懸念事項であるが、自動テストにも当てはまる。例えば、テストサイクルが短く、長時間実行が必要な自動テストが多い場合である。このため、特定の基準に基づいて適切なリグレッションテストケースを選択する必要がある。リグレッションテストの範囲は、テストサイクルごとに見直すことが不可欠である。見直しの結果、既存のリグレッションテストスイートを調整する必要があると結論づける場合がある。

自動テストを選択するための最も信頼できる技法は影響度分析であり、ツールはこれを支援することができる。このようなツールは、自動化された構成管理システムに基づいている。ツールは、各テストケースの実行中に、どの構成アイテムがアクティブになったかを記録する。変更が発生すると、どの構成アイテムが修正されたかを追跡し、その構成アイテムと相互作用するリグレッションテストを選択する。こうすることで、構成アイテムの変更後、故障を発見する可能性が最も高い部分にテストが集中することを確かにする(Juergens et al.)

手動でテストを実行する場合、リグレッションテストの選択において明確に優れている技法があると結論づけられるだけの決定的な根拠は、これまで示されていない。というのも、その結果はさまざまな要因に左右されるためである(Engström et al.)。したがって、TAは、与えられた状況に基づいて、どの技法を使用するかを判断しなければならない。一般的に使用される技法は、以下の通りである:

- **リスクベースのテスト選択**では、リグレッションテストスイートとリスクレジスターとのトレーサビリティをTAが維持する。変更が行われ、それに応じてリスクレジスターを更新した場合、TAは最も高いリスクレベルをカバーするようにリグレッションテストスイートを調整する。
- **履歴ベースドテスト**では、TAが過去のテスト実行を評価し、その中から、欠陥を検出したもの、または前回のテスト実行以降に加えられた変更と類似の変更で影響を受けやすかったものを判別する。リグレッションテストの一部として、対応するテストを再度実行することで、同様の欠陥を検出する可能性が高まる。TAはまた、長い間実行していないテストを含め、それらが引き続き合格するかを確認することもできる。
- **カバレッジベースドテスト**では、TAが選択したテスト技法に基づいて、できるだけ少ないテストで高いカバレッジを達成するようにテストを選択する。テスト数は、テスト1件あたりで得られるカバレッジの増分とのバランスを慎重に取らなければならない。
- **要件トレーサビリティマトリクス**では、要件の変更が関連するテストに与える影響を評価するために使用する。これは、新しい要件や変更された要件が間接的にテストに影響する場合に特に有用である。TAは、意図しない副作用の可能性をカバーするために、直接影響を受ける機能と関連する機能のリグレッションテストを選択する。アジャイルソフトウェア開発では、新しいユーザーストーリーや変更されたユーザーストーリーによって影響を受ける受け入れ基準をカバーするテストを選択することによって、同様に行うことができる。
- **運用プロファイルに基づくテスト**では、TAがテスト対象の使用パターンに基づき、実行するリグレッションテストケースを選択する。例えば、オンラインストアをテストするとき、そのようなテストの1つとして、ユーザーがログインし、商品を検索し、カートに追加し、注文を行うといった一連の流れを含めることができる。アプリケーションを大幅に変更する場合、この技法により、システム全体の機能を素早く概観できる。この結果、テストが多くなりす

©International Software Testing Qualifications Board

ぎる場合、TAは、頻繁に発生し重要な機能とビジネスプロセスをカバーする、重要な使用パターンに優先順位を付ける。

- **影響度分析**は、変更された構成アイテムと相互作用するリグレッションテストケースをTAが知っていれば、手動で実行するリグレッションテストケースの選択に使用することもできる。

より包括的で効果的なリグレッションテストスイートのために、選択技法を組み合わせて使用することが必要となることが多い。しかし、TAは、カバレッジの必要性とテストスイートのマネジメント可能なサイズのバランスを注意深く取らなければならない。各テストサイクル終了後、TAはテスト結果を分析し、適用した技法の有効性を判断する(*5.3.1 項* 参照)。次回、TAは効果的な技法を維持し、効果的ではない技法を置き換える。これによって、リグレッションテストの選択を継続的に改善していく。これは、変更が頻繁なイテレーティブ開発モデルやインクリメンタル開発モデルにおいて不可欠である。

©International Software Testing Qualifications Board

3 テスト分析とテスト設計 - 615分

キーワード

チェックリストベースドテスト、振る舞いに基づくテスト技法、組み合わせテスト、クラウドテスト、CRUDテスト、データに基づくテスト技法、デシジョンテーブルテスト、ドメインテスト、同値パーティション、経験ベースのテスト、メタモルフィック関係、メタモルフィックテスト、ランダムテスト、ルールに基づくテスト技法、シナリオベースドテスト、セッションベースドテスト、状態遷移テスト、テストチャーター

第3章の学習の目的

3.1 データに基づくテスト技法

TA-3.1.1 (K3) ドメインテストを適用する。

TA-3.1.2 (K3) 組み合わせテストを適用する。

TA-3.1.3 (K2) ランダムテストの利点と制限について要約する。

3.2 振る舞いに基づくテスト技法

TA-3.2.1 (K2) CRUD テストについて説明する。

TA-3.2.2 (K3) 状態遷移テストを適用する。

TA-3.2.3 (K3) シナリオベースドテストを適用する。

3.3 ルールに基づくテスト技法

TA-3.3.1 (K3) デシジョンテーブルテストを適用する。

TA-3.3.2 (K3) メタモルフィックテストを適用する。

3.4 経験ベースのテスト

TA-3.4.1 (K3) セッションベースドテストのテストチャーターを準備する。

TA-3.4.2 (K3) 経験ベースのテストを支援するチェックリストを準備する。

TA-3.4.3 (K2) クラウドテストの利点と制限の例を挙げる。

3.5 最適なテスト技法を適用する

TA-3.5.1 (K4) ある状況に対してプロダクトリスクを軽減する適切なテスト技法を選択する 。

TA-3.5.2 (K2) テスト設計の自動化の利点とリスクを説明する。

©International Software Testing Qualifications Board

テスト分析とテスト設計について

テスト技法は主にテスト分析とテスト設計で使用する。本章で取り上げるテスト技法は、ブラックボックステスト技法と経験ベースのテスト技法である。ホワイトボックステスト技法については、(ISTQB-TTA v4.0)で述べる。

本シラバスでは、ブラックボックステスト技法を、モデル化するテスト条件に基づき、3つのカテゴリーに細分化する:

- データの要素(データに基づく)
- 動的振る舞いの要素(振る舞いに基づく)
- 静的振る舞いルールの要素(ルールに基づく)

3.1 データに基づくテスト技法

本節では、テストアイテムの入力データの集合を表すために「ドメイン」という用語を使用する。ドメインのさまざまな領域からの入力データは、テストアイテムのさまざまな振る舞いにつながる。データに基づくテスト技法は実装が特定のドメイン領域を正しく処理することを検証することを目的とする。Foundation Levelシラバス(ISTQB-CTFL、v4.0.1、4.2節)では、同値分割法(EP)と境界値分析(BVA)という、データに基づく2つのテスト技法を取り上げている。本シラバスでは、さらに3つのデータに基づくテスト技法を紹介する:

- ドメインテスト - EPとBVAを、複数のパラメーターと複雑なパーティションを持つドメインに拡張する。
- 組み合わせテスト - 多次元ドメインにおける複数のパラメーターの相互作用に焦点を当てる。
- ランダムテスト - 指定された確率分布に基づいて、ドメインからランダムな入力を選択する。

ランダムテストは一般に、ランダムな入力データとランダムなイベントの両方を指すことに注意すること。本シラバスでは、ランダム入力データでのテストのみを扱う。

3.1.1 ドメインテスト

ドメインテストは、テストアイテムがドメインの同値パーティションおよびその境界線において、仕様どおりに振る舞うかを検証する。この場合、同値パーティションは、ブール演算子(AND、OR、NOT など)で不可分条件の組み合わせと、相互作用する1つ以上の変数を含む式を用いて定義する。各不可分条件は、同値パーティションの境界線を定義する。≤、≥、＝を用いた関係式が閉じた境界線を形成し、＜、＞、≠を用いた関係式が開いた境界線を形成する。

例えば、「身長＞1.29メートル AND 体重／身長^2≥ 30」という式は、2つの変数の二次元領域の同値パーティションを定義するもので、開いた境界線と閉じた境界線の2つを持つ。

ドメインテストは、同値パーティション実装の欠陥(例:演算子や定数の誤り)を、その欠陥を明らかにできる適切なカバレッジアイテムを選択することで識別しようとするものである。ドメインテストにおけるカバレッジ基準とは、ONポイント、OFFポイント、INポイント、OUTポイントを指す:

- 閉じた境界線の場合、ONポイントはこの境界線上にある。開いた境界線では、ONポイントは同値パーティションの内側にあり、与えられた精度に従って境界線に最も近い。
- 閉じた境界線では、OFFポイントは同値パーティションの外側にあり、与えられた精度に従って境界線に最も近い。開いた境界線では、OFFポイントはこの境界線上にある。
- INポイントは同値パーティションに属し、ONポイントではない値となる。
- OUTポイントは同値パーティションの外側にあり、OFFポイントではない値となる。

©International Software Testing Qualifications Board

これらのポイントのタイプはそれぞれ、同値パーティションの境界線に関係している。同じポイントであっても、異なる境界線に対して異なるタイプを持つことができる。

カバレッジ基準は以下の通りである:

簡易ドメインカバレッジ（Jeng et al., 1994）は、以下のカバレッジアイテムを必要とする：
- 演算子 <、≤、>、≥ のいずれかで定義される各境界線について、ONポイントを1つ、OFFポイントを1つ。
- 演算子 = の境界線について、ONポイントを1つ、境界線の両側に位置するOFFポイントを2つ。
- 演算子 ≠ の境界線について、OFFポイントを1つ、境界線の両側に位置するONポイントを2つ。

各OFFポイントは、与えられた精度に従って、対応するONポイントに可能な限り近くなければならない。

信頼性の高いドメインカバレッジ（Forgács et al, 2024）は、以下のカバレッジアイテムを必要とする：
- 演算子 <、≤、>、≥ のいずれかで定義される各境界線について、ONポイントを1つ、OFFポイントを1つ、INポイントを1つ、OUTポイントを1つ。
- 演算子 = の境界線について、ONポイントを1つ、境界線の両側に位置するOFFポイントを2つ。
- 演算子 ≠ の境界線について、OFFポイントを1つ、境界線の両側に位置するONポイントを2つ。

ここで議論した両カバレッジ基準について、カバレッジアイテムの数は境界線の数に線形に依存する。両カバレッジ基準では、異なる境界線で同じカバレッジアイテムを用いることで最適化できる。例えば、同値パーティションのすべての境界線で共通のINポイントを用いることができる。また、ある境界線のONポイントとOFFポイントの組を、隣接する同値パーティションの境界線に対してはOFFポイントとONポイントとして用いることもできる。境界線が多数あるドメインに対しては、高度な最適化アルゴリズムが利用できる（Forgács et al,2024）。

信頼性の高いドメインカバレッジは、簡易ドメインカバレッジよりもカバレッジアイテムが若干多くなるが、かなり多くのドメイン欠陥を検出することができる（*Site of Software Test Design*, 2020）。

ドメインテストは、BVAとEP（ISTQB-CTFL、v4.0.1、4.2節を参照）をより複雑なドメインに対して一般化したものであり、どのテストレベルでも適用できる。ドメインテストに対するさまざまなアプローチは、次のような教科書で確認できる(Beizer, 1990, Ch. 6; Binder, 2000, Section 10.2.4; Kaner; Padmanabhan, et al., 2013; Jorgensen, 2014, Ch. 5).

3.1.2 組み合わせテスト

ソフトウェアの故障の中には、パラメーター値の特定の組み合わせから発生するものがあり、相互作用の故障として知られている。組み合わせテストは、パラメーター値の組み合わせを探索することによって、このような故障を明らかにすることを目的としている。

組み合わせテストでは、テスト条件は通常、構成パラメーターまたは入力データを組み合わせる。したがって、組み合わせテストには主に2つのアプローチがある。第一のアプローチは、構成パラメーターの組み合わせを使用し、これらの組み合わせのそれぞれを、同じテストケースを使用してテストすることができる。2つ目のアプローチは、入力データ値の組み合わせを使用するもので、これらは完全なテストケースの一部となり、テスト対象システムのテストスイートを作成する（D. R. Kuhn et al.,2013）

特定のパラメーター - 値ペアは、パラメーターとその値で構成される（例えば、'(色,赤)'）。

組み合わせカバレッジ基準には以下のものがある（Ammann et al., 2008、）（Forgács et al.,2019）

- **ベースチョイスカバレッジ**は、いくつかのパラメーター - 値ペアが、他よりも重要であると仮定する。ベースカバレッジアイテムは、各パラメーターにてベースとなるパラメーター - 値ペアを選び、それらを組み合わせたものである。後続のカバレッジアイテムは、各パラメーターについて、ベースカバレッジアイテム中のベースのパラメーター - 値ペアを、ベースではない各値で置き換えて作成する。
- **ペアワイズカバレッジ**では、カバレッジアイテムは任意の2つのパラメーターに関するパラメーター - 値ペアの組み合わせである。カバレッジアイテムを生成するためにツールが利用できるが、ペアワイズカバレッジを達成する最小のテストケース集合を見つけることは、一般に困難である。

©International Software Testing Qualifications Board

多くの値を持つパラメーターについては、まずEPを適用して、その数と、結果として得られる組み合わせの集合を減らすことができる。パラメーターとその値をクラシフィケーションツリーまたはフィーチャーモデル（IREB-Glossary, 2024 を参照）にまとめておくことは、この活動を支援する。最終的なテストケースの数は、パラメーター-値ペア間の制約、問題のあることがわかっているため手動で追加した組み合わせ、あるいは無効な組み合わせや実行不可能な組み合わせによって影響を受ける場合がある。

組み合わせテストにとって重要な洞察は、すべてのパラメーターが故障に寄与するわけではなく、ほとんどの故障は単一のパラメーター値または比較的少数のパラメーター間の相互作用によって引き起こされるということである（Cohen et al.,1994）。これは、プログラムにおける単純な欠陥を検出することで、複雑な欠陥も発見できることが多いとする結合効果仮説と整合する（Offutt, 1992）。限られた範囲の調査（D. Kuhn et al., 2004）では、故障の約97%が1つか2つの相互作用する条件によってのみ引き起こされることが示されており、ペアワイズテストが効果的なテスト技法であることを示している。

diff-pair-t や n-wise テストなどの他のカバレッジ基準を含む組み合わせテストに関する追加情報は、（Ammann et al., 2008）および（Forgács et al., 2019）に示されている。組み合わせテストを支援するツールについては、（Czerwonka, 2004）でも入手できる。

3.1.3 ランダムテスト

ランダムテストでは、指定された確率分布に基づき、テストアイテムの入力ドメインからテストデータをランダムに選択する。妥当性確認の目的では、運用プロファイルに基づく分布が推奨される。検証の目的では、バイアスを避けるために、分布は利用状況に依存しないものであるべきである。テストオラクルを使用して期待結果をテストケースに追加する場合があり、これには多くの場合、自動化したテストオラクルが必要となる。

ランダムテストには、ガイドありとガイドなしがある。ガイドなしランダムテストでは、確率分布はプロセス全体を通して一定のままである。適応的ランダムテスト（Huang et al., 2019）などの技法に代表されるガイドありランダムテストでは、これまでに選択した値に基づいて分布を調整し、時間の経過とともに変化させる。ガイドありランダムテストは、欠陥がドメインの特定領域に集中することが多い点を考慮し、入力ドメインを効果的にカバーすることを目的とする。

ランダムテストには、広く合意されたカバレッジ基準がない。したがって、終了基準は、実行したテスト数、テスト時間、または同様の完了度を示す尺度に基づくしかない。

ランダムテストは、ドメイン知識が限られている場合や、大量のテストデータが必要な場合に特に有用である。コスト効率が高く、確率的な観点からテスト対象の信頼性に関する示唆を与える。ランダムテストは、特定のコードや機能に対する誤った信頼により手動テストで欠陥を見落とす、といったバイアスを避けるのにも役立つ。しかしランダムテストには、データセマンティクスの軽視、データの意味に関わる欠陥の見逃しの可能性、特定の欠陥の見落とし、冗長なテストの生成、自動化したテストオラクルへの依存、ならびにランダムな出力によるテスト結果の不整合といった、いくつかの課題や制限もある。各テストコンテキストにおいて、ランダムテストの利点と制限のバランスを取ることが重要である。

従来、ランダムテストは他のテスト技法に比べて効果的ではないと考えられてきた。近年、この仮説は多くの実証研究で検証されている。上述の状況下では、ランダムテストは他のデータに基づくテスト技法よりも効果的かつ効率的になり得ることが示されている (Arcuri et al., 2012), (Wu et al., 2020)。ランダムテストは、ファズテストやカオスエンジニアリングにも適用されている。

3.2 振る舞いに基づくテスト技法

振る舞いに基づくテスト技法は、テストアイテムの動的な（すなわち、状態に依存する）振る舞いの仕様からテストケ

©International Software Testing Qualifications Board

ースを導出する。本シラバスでは、3つの振る舞いに基づくテスト技法について説明する:

- CRUDテスト
- 状態遷移テスト
- シナリオベースドテスト

CRUDテストとシナリオベースドテストは、(ISTQB-CTFL、v4.0.1、4.2節)にて既知のブラックボックステスト技法の範囲を拡張し、状態遷移テストについては、カバレッジ基準を追加する。

3.2.1 CRUDテスト

CRUD テストは、テストアイテムによって処理されるデータエンティティのライフサイクルを検証する。CRUDとは、*create (作成)*、*read (読み込み)*、*update (更新)*、*delete (削除)*の略である。これらは、機能がエンティティに対して実行できる4つの基本操作である。

CRUDマトリクスは、データエンティティのライフサイクルの概要を示す。列はエンティティを表し、行は機能を表す。ある機能が、指定されたエンティティに対して、作成、読み込み、更新、または削除といった1つ以上の特定の操作を実行するとしている。これは、操作の頭文字であるC、R、U、Dを用いてマトリクス上に示される。CRUDマトリクスを作成するために、TAは各機能について、4つの操作のうちどの操作をどのエンティティに対して実行するかを決定する。C-U-D操作に暗黙的にリンクされていることが多い読み込み操作には、特に注意を払うべきである。

CRUDテストは、2つの部分から構成される(Koomen et al., 2006)

- **完全性テスト**は静的テストであり、すべての可能な操作(すなわち、C、R、U、D)がすべてのエンティティに対して発生するかどうか(すなわち、すべてのエンティティについてライフサイクル全体が実装されているか)を検証する。ある操作が存在しないことは、調査が必要な不正である。
- **一貫性テスト**は、動的テストであり、さまざまな機能を統合し、エンティティが一貫して使用されているかを確認することを目的としている。エンティティを扱うときに、各機能が正しく相互作用することを検証する。テストケースは、CRUDマトリクスのすべての操作をカバーすべきである。さらに、ネガティブなテストも含めるべきである(例えば、まだ作成されていないエンティティの読み込み)。テストケースは、ライフサイクル全体をカバーするように機能を組み合わせて、エンティティごとに設計する。

CRUDカバレッジは、テストスイートが実行した操作の数をCRUDマトリクスの操作の総数で割った値で測定する。より厳密なCRUDカバレッジは、操作の特定の組み合わせをカバレッジアイテムとして考慮することができる(例:各Uの後に、可能なすべてのRをカバーする必要がある)。

CRUDテストは、主にシステムレベルで用いられる。データ整合性違反、アクセスコントロールの欠陥、あるいはデータの不整合など、エンティティの扱いにおけるテストアイテムの振る舞いに関する欠陥に焦点を当てる。

3.2.2 状態遷移テスト

複雑なシステムの多くは状態を持つ(つまり、あるイベントに対するシステムの反応は、その時点のシステムの状態に依存する)。状態を持つシステムの例としては、組み込みシステム、対話型システム、制御システム、またはエンティティとそのライフサイクルを扱うシステムが挙げられる。

状態は、複雑な内部の詳細を単純化し、ステークホルダーが期待動作を理解しやすくする。テスト分析中、TAは、状態に基づくモデルが、必要な詳細レベルでテストアイテムの期待される振る舞いを表していることを確認しなければならない。テストベースにすでにそのようなモデルが含まれている場合、TAは、そのモデルにテスト条件が含まれているかを確認し、必要であれば、モデルを調整するか、新しいモデルを設計しなければならない。

状態ベースモデルでは、ノードは状態を表し、エッジは遷移を表す。状態遷移はイベントによってトリガーされ、ガード条件やアクションを含むこともある(ISTQB-CTFL、v4.0.1、4.2.4項参照)。これらのモデルにはバリエーションがあり、拡張有限ステートマシン(Bochmann et al., 1994)、ハレルのステートチャート(Harel, 1987)、またはUMLステートマシン(*OMG® UML*,2017)などが利用されている。

©International Software Testing Qualifications Board

(ISTQB-CTFL, v4.0.1)で説明した状態遷移カバレッジ基準に加え、高い欠陥検出有効性が実証研究で示されている2つの基準を以下に説明する:

- Nスイッチカバレッジは、N+1個の連続する遷移の有効なシーケンスに適用され、Nスイッチとも呼ばれる(Chow, 1978)。0スイッチカバレッジは、有効遷移カバレッジに等しい(ISTQB-CTFL、v4.0.1、4.2.4項参照)。1スイッチとは、1つの状態における入出遷移のペアである。N-スイッチをその末端で有効な次の状態遷移に拡張すると、N+1スイッチとなる。0スイッチと1スイッチのカバレッジは、実際には頻繁に使用される。Nスイッチの数はNに比例して指数関数的に増加するため、2スイッチ以上での100%カバレッジは、イベントの予期しないシーケンスによって故障が起こるリスクが高い場合に限って適切である。
- ラウンドトリップカバレッジ(Ammann et al., 2008)は、ループを形成する状態ベースモデルのパスに適用する。ラウンドトリップとは、開始状態と終了状態が同じで、このループ内の他の状態が2回発生しないループのことである。このラウンドトリップがカバレッジアイテムとなる。(Antoniol et al., 2002) は、この基準が欠陥の発見に非常に有効であることを発見した。

状態遷移テストは、モデルベースドテストツールを使った自動化に適している。ほとんどのブラックボックステスト技法と同様に、状態遷移テストは、モデリング中に仕様書の欠陥を検出することで、欠陥予防に貢献する。状態遷移テストは、どのテストレベルでも適用できる。

さらなる状態遷移のカバレッジ基準については、(Rechtberger et al., 2022) で議論されている。近年提案されている状態に基づくモデルとしては、アクション - 状態モデルがある。(Forgács et al., 2024)。

3.2.3 シナリオベースドテスト

シナリオベースドテストは、現実的なシナリオの中でテストアイテムの振る舞いを評価する。ユーザーリサーチ、ユーザーストーリー、ペルソナ、ユーザージャーニーマップ(*第11章参照*)などの技法は、価値あるシナリオを識別するのに役立つ。シナリオベースドテストでは、TAは、テストアイテムを通過するワークフローを構成するアクションシーケンスに基づいて、シナリオモデルを作成する(*ISO/IEC/IEEE 29119-4*, 2021)。本シラバスでは、アクティビティ図とユースケースの2つのモデルについて説明する。その他のモデルには、フローチャート、ビジネスプロセスモデル＆ノーテーション図(*OMG® BPMN*, 2013)、シーケンス図、またはコラボレーション図(OMG® UML, 2017)がある。これらのモデルについては、本シラバスでは説明しない。詳細は(ISTQB- AcT, v1.0)を参照。

アクティビティ図とは、システム内のワークフローをグラフィカルに表現したものである。アクティビティ図は、ビジネスプロセスのモデリングに特に役立つが、コントロールフローをモデリングすることもできる。アクティビティ図では、フローチャートの記法を拡張し、並行性をモデリングできる。アクティビティ図の主な要素は、開始ノードと終了ノード、アクション、遷移、判定ノード、マージノード、フォークノード、ジョインノード、スイムレーンである。

ユースケースは、ユーザーとシステム、またはシステム間の相互作用を表す、アクションのテキストまたはグラフィカルな記述である。モデルによって3つのタイプのシナリオに区別できる:

- **メインシナリオ**(いわゆる「ハッピーパス」) - ユーザーの視点から特定のゴール達成につながる、典型的で期待される一連のアクション。ユースケースは1つのメインシナリオしか持つことができない。
- **拡張**(または代替シナリオ)-メインシナリオ以外の一連のアクションで、最終的にメインシナリオのゴール達成につながる。
- **例外** - 予期せぬアクション(異常な使用や無効な入力など)により、メインシナリオのゴールを達成できない一連のアクション。

シナリオベースドテストでは、TAは、シナリオ(すなわち、カバレッジアイテム)をカバーするテストケースを、多くの場合、リスクベースの優先順位に従って設計する。シナリオモデルにループがない場合、各シナリオを個別のテストケースでテストすることができる(すなわち、モデル内のすべての可能なシナリオをテストスイートで実行できる)。ループが存在する場合、潜在的なシナリオ(パス)の数は無限になり得る。この場合、シンプルループカバレッジをシナリオモデルに適用できる。シンプルループカバレッジでは、各ループが0回実行された場合(すなわち、スキッ

©International Software Testing Qualifications Board

プした場合)、ちょうど1回の反復で実行された場合、2回以上の反復で実行された場合(すなわち、典型的な反復回数)、および(可能であれば)最大反復回数で実行した場合のテストを必要とする。

シナリオベースのカバレッジは、実行したシナリオ数を識別した全シナリオ数で割った数で測定できる。シナリオは、シナリオモデルにさまざまなカバレッジ基準を適用することによって識別できる(Koomen et al., 2006)。カバレッジ基準によっては、各シナリオを複数回テストすることを求める場合がある。例えば、あるシナリオでは、そのシナリオ内に現れる変数に対して追加の EP または BVA のカバレッジが必要なことがあり、その場合、1つのシナリオを複数のテストケースでテストする必要があるかもしれない。

シナリオベースドテストは、システムテストや受け入れテストで実施することが多い。これは、ユーザーの視点から見たシステムの機能適合性(*4.1*節参照)に焦点を当てたエンドツーエンドテストの形を取る。しかし、シナリオベースドテストは、他のテストレベル(例えば、相互作用プロトコルに基づいたシナリオによるコンポーネント統合テストや、さまざまなメソッドを呼び出すシナリオによる状態を持つオブジェクト指向クラスのコンポーネントテスト)や、非機能テスト(例えば、シナリオは、信頼性、柔軟性、または互換性テストで使用される運用プロファイルの要素を構成することがある)でも使用することができる。

3.3 ルールに基づくテスト技法

ルールに基づくテスト技法は、テストアイテムの状態に関係なく有効なルール(例:ビジネスルール)によって仕様化した、テストアイテムの状態を持たない振る舞いの実装を検証する。本シラバスでは、2つのルールに基づくテスト技法を取り上げる:

- デシジョンテーブルテスト
- メタモルフィックテスト

3.3.1 デシジョンテーブルテスト

Foundation Levelのシラバス(ISTQB-CTFL、v4.0.1)は、デシジョンテーブルテストの基本をカバーしている。本シラバスでは、より高度なトピックについて説明する。使用する用語と表記法は、標準(*OMG® DMN*, 2024。)に準拠する。

デシジョンテーブルテストでは、TAは通常、完全なデシジョンテーブルを作成するか、テストベースから得られた既存のデシジョンテーブルを分析することから始める。完全なデシジョンテーブルのルール数は、その条件の値の数の積である。この数は、条件の数とその値の数とともに指数関数的に増大し、デシジョンテーブルを最小化する動機となる。最小化は、元のデシジョンテーブル(完全なものであってもなくてもよい)から開始し、より少ないルールを持つ等価なデシジョンテーブルを導出する。

デシジョンテーブルは、don't care演算子「-」を使用してルールをマージすることで、最小化することができる。ルールをマージするとき、実行不可能なルール(すなわち、絶対に発生しない条件値の組み合わせを持つルール)は無視することが推奨される。多くの場合、実行不可能なルールは、マージする前にデシジョンテーブルから削除する。考えられる体系的な最小化アルゴリズムでは、アクションが等価でありながら、ある1つの条件だけが異なり、その条件にて取りうる値をすべてカバーしているルールを探す。これらのルールをマージし、異なる条件値は「-」で置き換える。体系的な最小化の結果は、列を最小化する順序に依存することがあり、常に最適解を導くとは限らない。TAは、さらなる最小化が可能かどうかを確認しなければならない。

場合によっては他のステークホルダーとともに、デシジョンテーブルを以下の基準でレビューすることがTAの仕事である:

- 一貫性(すなわち、2つの異なるルールが同じ条件値の組み合わせに適用される場合、それらはアクションが等価であること)
- 実現可能性(すなわち、実現不可能なルールを含まない)
- 完全性(すなわち、実現可能な条件値の組み合わせが欠落していないこと)
- 正確性(すなわち、ルールがシステムの意図した振る舞いをモデル化していること)

©International Software Testing Qualifications Board

加えて、ルールは重複しないことが望ましい(すなわち、どのような条件値の組み合わせに対しても、適用されるルールは最大でも1つである)。ルールの重複は、元のデシジョンテーブルがすでに最小化されている場合や、ルールのマージが正しくない場合に発生する可能性がある。

チェックサムプロシジャーは、重複とギャップを示すために、最小化されたデシジョンテーブルのルール数を使用する。最小化された表の各ルールについて、元のデシジョンテーブルのルールの数を計算する。条件に「-」がないルール(すなわち、すべての条件に個別の値があるルール)のスコアは1である。「-」があるごとに、その条件に対応する個別値の数をルールのスコアに乗じる。ルールのスコアの合計が、最小化されたデシジョンテーブルのチェックサムとなる。チェックサムが元のデシジョンテーブルのチェックサムより小さい場合、最小化されたデシジョンテーブルは不完全である。チェックサムの方が大きい場合、いくつかのルールが重複しているか、追加のルールが存在する(例えば、実行不可能な条件の組み合わせ)。最小化が正しく実行された場合、チェックサムは等しくなる。チェックサムが等しいだけでは、最小化されたデシジョンテーブルが元のデシジョンテーブルと等価であることを保証しない。

デシジョンテーブルのカバレッジは、実行された列の数をデシジョンテーブルの実行可能な列の総数で割ることによって測定できる。デシジョンテーブルのルールから生じるテストケースを実装する場合、TAは条件とアクションを実装しなければならない。「-」は少なくとも2つの値を表すため、TAは、与えられたルールの「-」条件値をどのように実装するかを決定しなければならない。しかし、デシジョンテーブルに関連するリスクレベルが高い場合、TAは最小化を控えるべきであり、完全なデシジョンテーブルの実行可能な列のデシジョンテーブルカバレッジを測定すべきである。

3.3.2 メタモルフィックテスト

メタモルフィックテスト(MT)は、既存のソーステストケースを基にテストケースを生成することを目的とした技法である。1つ以上のフォローアップテストケースは、メタモルフィック関係(MR)に基づいてソーステストケースを変更(メタモルフィック化)することによって生成される。MRはテストアイテムの特性を定義し、テストケースの入力の変更がどのようにテストケースの期待結果に反映されるかを記述する。

TAは、MRのソーステストケースとフォローアップテストケースをテストプロシジャーに結合し、合同でテスト結果を評価する。それらがメタモルフィック関係を満たしていればテストは合格し、そうでなければ不合格となる。故障の場合、後続のデバッグは、関係する個々のテストケースのどれが不合格となったかを判断しなければならない。

例えば、一連の数値の平均を求める関数を考えてみよう。一連の数値と予想される平均値でソーステストケースを作成し、テストケースを実行して合格を確認する。MRは、一連の数値のどのような並べ替えでも同じ平均になることを示すことができる。

このMRを使って、TAは同じ数値の集合を入力に持ちながら順序だけが異なるフォローアップテストケースをいくつか作成することができる。期待結果は変わらない。

同じ平均関数に対して、TAは別のMRを使用することもできる。そのMRは、数列の各数値を同じ数 x で掛けた場合、期待される結果(平均値)も x 倍になるというものである。このMRを使うことで、TAは x の値を変えるだけで、1つのソーステストケースから任意の数のフォローアップテストケースを作成できる。これは、テスト設計やテスト実行が自動化されている場合に特に有用となる。さらにTAは、2つ以上のMRを組み合わせて、フォローアップテストケースを作成することもできる(例えば、順列を付けて2倍にする)。

MTは、テストオラクル問題がある場合にも使用できる(*1.3.4*項参照)。このような状況では、ソーステストケースとフォローアップテストケースの期待結果が得られないため、それらのテスト結果を個別に評価することはできない。例として、大規模なデータセットに基づいて死亡時年齢を予測する、AIベースの保険数理プログラムを挙げることができる。MRは、喫煙本数が増えれば、予測される死亡年齢が下がるはずだと述べるかもしれない。

現在のところ、有用な終了基準となるMTのカバレッジ尺度は認められていない。各MRを1回ずつカバーするだ

©International Software Testing Qualifications Board

けでは、期待結果の部分的な検証しかできないため、不十分である。TAは、MTをランダムテストと組み合わせて、同じMRに対して多くのローレベルソーステストケースとフォローアップテストケースを生成することができる。例えば、前述の平均計算では、乱数発生器を使用してソーステストケース用のさまざまな入力を生成し、フォローアップテストケース用のランダムな順列と乗数を生成することができる。

MTはほとんどのテストアイテムに使用でき、機能テストと非機能テストに適用できる(例えば、負荷テスト、MRを使用したフォローアップテストケースによる負荷生成、複数のシーケンスで選択できるさまざまなインストールパラメーターを使用したインストール性テストなど)。AIベースのシステムで推奨されるテスト技法である(ISTQB-AI, v1.0)。

詳細については、標準(*ISO/IEC/IEEE 29119-4*, 2021)または調査論文(Segura; Towey, et al, 2020)および(Segura; Fraser, et al, 2016)も参照のこと。

3.4 経験ベースのテスト

TAは、経験ベースのテストアプローチとテスト技法を採用し、専門知識と過去の経験をテストの指針として活用する。本節では、セッションベースドテストとチェックリストベースドテスト(ISTQB-CTFL、v4.0.1、4.4.2 項と 4.4.3 項を参照)における、TAによるドキュメンテーションの利用について詳述する。さらに、クラウドテストについても説明する。経験ベースのテスト技法はすべて、受け入れテスト駆動開発のようなコラボレーションベースのテストに適用することができる。詳細については、(ISTQB-CTFL、v4.0.1、4.5節)を参照のこと。

3.4.1 セッションベースドテストを支援するテストチャーター

探索的テストでは、テストチャーターはテストセッションのミッションを示し、その範囲と目的、制限、スケジュール、リスクなどの情報を概説する。これはテストのロードマップとして機能し、テストセッションに構造を提供する。テストチャーターは、TAがテストすべき特定の領域やフィーチャーに集中するのを助けると同時に、必要なときにはシステムを柔軟に探索できるようにする。テストチャーターは、各テストセッションで実行するテストスイートを指定するものではない。

セッションベースドテストのテストチャーターを作成する際、TAは、特にテストチャーターの設計に影響する特定の要素を考慮しなければならない:

- 顧客および要件の要素(例:顧客から引き出した要件、ビジネスユースケース、品質要件)、およびユーザージャーニーマップ(すなわち、時系列的なユーザーとシステムとの相互作用)。
- プロダクト要素(例:機能フロー、プロダクトの主なゴール、プロダクトのフィーチャー、ソフトウェア設計、インターフェース)
- プロジェクトマネジメント要素(例:時間的制約、プロジェクトの意義、見積もり工数、ビジネス価値など)

テストチャーターは、テスト目的とさまざまな追加情報を記述したミッションから構成される。

ミッションの一般的な軽量フォーマットは、「[target]を探索する／[resources]を用いて／[information]を発見するために」である(Hendrickson,2013)。ここで[target]は何を探索するか(例:領域、フィーチャー、リスク、コンポーネントおよび要件)を記述し、[resources]はTAが使用するもの(例:テストデータ、構成、ツール、制限、ヒューリスティック、依存関係)を記述し、[information]はTAがどのようなタイプの情報(例:品質特性評価、予想される欠陥のタイプ、標準違反)を発見することを狙っているかを説明する。

テストチャーターには、次の追加情報が含められるが、これらに限らない(Ghazi et al, 2017):

- 運営情報(例:テストセッションの所要時間、開始日時、テスト担当者名)
- テスト目的(例:テストの動機やテストチャーターのミッション)
- テストスコープ(例:テスト対象システム内の特定の関心分野、テストレベル、使用するテスト技法、テストのアイデア、終了基準、優先度、テストチャーターがカバーすべき範囲、テストしない内容の記述)
- 開始基準(すなわち、テストセッションを開始するために満たすべき事前条件)

©International Software Testing Qualifications Board

- プロダクト関連情報(例:定義、データ、コンポーネント間のワークフロー、システムアーキテクチャー)
- 制限(すなわち、プロダクトが絶対にやってはならないこと)
- テスト環境の記述
- テストに役立つ既存のデータソース、プロダクト情報、テストツール
- 履歴情報(例:互換性や相互運用性の欠陥のような過去に発見された欠陥事項、既存の異常に言及する現在の未解決問題、過去のテスト関連の故障パターン)
- 制約とリスク(例:使用される規制、規則、標準)

テストセッション中、TAはテスト結果とともに、質問、観察、将来のテストに関するアイデアなどを含むテスト結果記録作業を行う。このデータはセッションシートに記録する。

特定のテストチャーターに含まれる情報の範囲と詳細はさまざまであり、TAの柔軟性の程度に影響する。例えば、一般的なテスト目的のみを定義すれば、十分な検討の余地があるが、使用するテスト技法に関する追加すれば、TAが制約される可能性がある。一方、追加情報(システムが絶対にできないことなど)は、偽陽性結果を報告する可能性を低くできる。

3.4.2 経験ベースのテスト技法を支援するチェックリスト

チェックリストベースドテストは、その適応性、簡便性、ソフトウェア品質を保証する有効性から、広く使われているテスト技法である。チェックリストを使用することで、TAは、テストアイテムの既知のすべての重要な側面を確実にカバーすることができ、重要な領域の見落としを防ぐことができる。また、チェックリストは、テストサイクルやさまざまなTA間で一貫性をもたらす。チェックリストを使用する場合、TAは重要な側面に集中する。チェックリストは、TAが過去に経験した故障や欠陥を記録するものである。チェックリストは、TAが(探索的テスト中などに)アイデアを使い果たしたときに、思い出したり、インスピレーションの源となったりする。TAは、標準的なチェックリストや仲間の誰かが作成したチェックリストを再利用することで、時間を節約できる。ただし、これらのチェックリストがテストアイテムに関連している場合に限る。チェックリストは、テストケースの文書化に必要な作業の軽減に役立つ。これは、要件やソフトウェアが常に変化している場合には、大きな財産となる。

チェックリストの作成は、多くの場合TAの責任である。チェックリストは、テストの整理や構造化を助け、ガイドとして役立つため、経験ベースのテストを支援する。再利用可能で、保守性があり、明確で、効率的なチェックリストを作成するには、努力が必要である。

以下のステップは、経験ベースのテストに適切なチェックリストを設定するためのガイドラインとなる:

まず、TAはチェックリストの範囲、目的、形式を決定する。なぜなら、これらはテストの深さと要求される詳細レベルに影響するからである。実行チェックリストには、あるプロセスについて考慮すべき主要な要素が含まれている(例えば、チェックリストの項目には、チェックすべき入力フィールドの具体的な無効入力が含まれている)。確認するチェックリストは、思考プロセスを導く補助として機能し、アプリケーションをさらに調査するための経験ベースのテストアイデアを提供する(例えば、チェックリスト項目は、検索結果に関連性があるかどうかを確認することを要求するかもしれない)。

次に、TAはチェックリスト項目を定義するために必要な情報を収集する。これには、経験豊富な専門家からの情報収集、欠陥ライブラリや欠陥分類法(Beizer, 1990)、(Kaner; Falk, et al., 1999)の閲覧、関連文書のレビュー、リスク、テストケース、潜在的なシナリオの解析などが含まれる。チェックリスト項目は、明確で、具体的で、曖昧さがなく、一貫性があり、関連性があり、保守性があり、実行可能で、測定可能でなければならない。チェック項目は、「はい」、「いいえ」、または「該当しない」で答えられる質問として作成するべきである。重要度、潜在的影響、リスクレベルに基づいて優先度を割り当てる必要がある。チェックリストは、包括的なハウツーガイドではない。むしろ、専門家のための迅速でシンプルなツールである。

最後に、TAは、チェックリスト項目を、機能エリア、ユーザーの役割、テストレベル、またはその他の関連する基準に基づいて、論理的なグループに分類することによって、チェックリストを構造化し、整理する。別々のカテゴリー

©International Software Testing Qualifications Board

を作成することは、長いチェックリストには特に有効である。

可能であれば、テンプレートや標準を利用すべきである。TAは、業界標準や実績のある実践例に沿った既存のテンプレートや定義済みのチェックリストを利用することで、労力を節約することができる。

チェックリストは決して最終的なものではない。TAは継続的なレビューを行い、新しい発見事項、優先度の変更、他のテスト担当者からのフィードバック、以前のテストサイクルから得た教訓などを反映させ、洗練させていく。チェックリストを他のテスト担当者と共有することで、TAは一貫性と協調性を促進し、テストアイテムやテスト中に焦点を当てるべき重要な領域をよりよく理解することができる。

3.4.3 クラウドテスト

クラウドテストは、多様な場所にて多様な背景を持つ内部または外部のテスト担当者のグループにテストを分担するものである。これは、使用性の妥当性を確認するためのコスト効率の高い方法になり得て、機能的品質特性と非機能的品質特性の両方をカバーする(Alyahya, 2020)、(Leicht et al., 2017)。

クラウドテストの利点には以下のようなものがある:

- **多様なテスト環境** テスト担当者は、さまざまな地理的な場所にいて、さまざまなデバイス、ブラウザー、ネットワーク条件で、さまざまな環境設定を使用することができる。
- **高い柔軟性** 短時間に多数のテストを実施する必要がある場合でも、容易にスケールできる。
- **費用対効果** クラウドテストは通常、大規模で多様な社内テストチームを維持したり、外部のテストサービスで補ったりするよりも低コストである。
- **迅速なフィードバック** テスト担当者は素早くフィードバックの提供ができるため、故障の早期発見と修正に役立つ。
- **実際のユーザー視点** テスト担当者は、アプリケーションの実際のユーザーである場合もあり、ユーザーエクスペリエンスと使用性に関してよりよい洞察を提供することができる。これは、ユーザー受け入れテストにおいて特に価値がある。
- **変動性** テストは毎回さまざまなテスト担当者によって実行されるため、再現性が低くなる。これは制限とも言えるが、カバレッジが広くなり、欠陥の発見を増やすことにもなる。

クラウドテストの制限には、以下のようなものがある:

- **テスト品質の不安定性** テスト品質は、個々のテスト担当者のスキルによって大きく変わる可能性があるが、例えば、ユーザーエクスペリエンスに関するフィードバックが目的である場合には、これは関係ないかもしれない。
- **コミュニケーションの課題** タイムゾーンや文化の違い、言葉の壁など、さまざまな場所にいる多くのテスト担当者と調整するのは難しいことがある。
- **セキュリティ** ソフトウェアを外部のテスト担当者と共有することは、データセキュリティや機密性のリスクをもたらす。適切な対策によって、これらのリスクを軽減し、機密情報を開示したり盗用を助長したりすることなく、責任ある形でクラウドテストを利用できる。
- **文書化と報告** 大規模で多様なテスト担当者を相手にする場合、重複や偽陽性を含む多数の発見事項をマネジメントしつつ、網羅的なテスト文書を確保するのは困難な場合がある。

クラウドテストは、TAがテスト技法を適用する活動に置き換わるものではないが、多様なテスト環境におけるカバレッジを高めることができるアプローチである。

3.5最も適切なテスト技法の適用

テストは、特定のコンテキストにおいて、可能な限り効果的かつ効率的でなければならない。この目的のために、TAは、最も適切なテスト技法を選択する中で、テストマネージャーを支援する。さらに、TAはテスト活動を支援するために自動化を活用できる。これには、本節で説明するテスト設計の自動化と、1.3.6項で説明するテスト実行自動化の

©International Software Testing Qualifications Board

支援が含まれる。

3.5.1 プロダクトリスクを軽減するテスト技法の選択

最も適切なテスト技法を選択することは、効果的かつ効率的なプロダクトリスク軽減にとって極めて重要であり、以下を含む多くの要因に影響される。

テスト目的(ISTQB-CTFL, v4.0.1, 1.1.1項参照)は、テスト対象のどの側面を評価するかを指定する。これらはテスト技法の選択の指針であり、システムのタイプに依存する(例:エンジニアリングにおける数値計算にはドメインに基づくテスト、対して与信リスクマネジメントではデシジョンテーブルテスト)。

プロダクトリスクは、潜在的欠陥と関連している。これらの欠陥は、特定のテスト技法を用いて検出するのが最善であり、ほとんどのテスト技法は特定のタイプの欠陥の検出に焦点を当てている(本シラバスの*3.1*節、*3.2*節、*3.3*節、*3.4*節を参照)。例えば、次のようなものである:

- データに基づくテスト技法は、データ処理、入力ドメインの実装、ユーザーインターフェース、計算、パラメーターの組み合わせにおける欠陥を検出することができる。
- 振る舞いに基づくテスト技法は、機能の欠落、コミュニケーション上の欠陥、処理の欠陥など、ユーザー要件の欠陥を検出することができる。
- ルールに基づくテスト技法は、ロジックやコントロールフローの欠陥を検出することができる。

リスク分析は、適切なテストアプローチの決定にも役立つ。例えば、次のようなものである:

- カバレッジに基づく終了基準。リスクレベルが高いほど、より厳密なカバレッジが必要となる場合がある(例:ペアワイズカバレッジではなく、組み合わせカバレッジにおける全組み合わせ)。しかし、TAは常にカバレッジの強さと必要なテスト工数のトレードオフを考慮しなければならない。
- 経験ベースのテストは、リスクレベルが低い場合、カバレッジの定義が難しい場合、もしくはプロジェクトスケジュールがタイトな場合に使用できる。

テストベース テスト対象の仕様書がモデルを使用している場合、TAはこれらのモデルにテスト技法を使用することができる。テストベースからテストオラクルを導出することが不可能または困難な場合は、メタモルフィックテストや経験ベースのテスト技法を適用することができる。

繰り返し発生する欠陥タイプに関する知識があれば、その種の欠陥の検出に焦点を当てたテスト技法(例えば、チェックリストベースドテスト)を選択すべきだと判断できる場合がある。以前のイテレーションやプロジェクトと同じような欠陥の発見を期待するのであれば、過去に成功したのと同じテスト技法を使用することが合理的かもしれない。

テスト担当者の知識と経験 TAがあるテスト技法に精通していない場合、重要なテスト課題においてそのテスト技法を使用することは推奨されない。ドメインの知識も、テスト技法の選択に影響を与えるかもしれない。例えば、ドメインの知識がほとんどない、あるいはまったくない場合、探索的テストのようなテスト技法は効果がないかもしれない。

使用されるソフトウェア開発ライフサイクル シーケンシャル開発モデルは、より形式的な技法を採用するのに適している。対照的に、イテレーティブ開発モデルは、軽量なテスト技法(例えば、経験ベースのテスト技法)を採用する場合や、テスト設計を自動化できる場合に適しているかもしれない。

顧客および契約要件 契約は、特定のテスト(例えば、特定のテストレベルやテストタイプ)の実行を明示的に要求することができ、これはテスト技法の選択に影響を与える(例えば、顧客から提供された一連のシナリオによる受け入れ基準は、シナリオベースドテスト技法の使用を示唆する)。

規制要件 プロジェクトが標準に従う場合、特定のテスト技法の使用を要件とすることがある。例えば、標準(ISO 26262, 2018)では、テスト対象に割り当てられたASIL(自動車用安全度水準)に応じて、同値分割法、境界値分析、エラー推測などのテスト技法の使用を要件としている。

プロジェクトの制約 時間や予算などの制約は、時間のかかる技法や高価なリソースを必要とする技法の適用に影響を与える場合がある。

欠陥検出の効率性と有効性を高めるために、テスト技法を組み合わせることが多い。例えば、以下が挙げられる:

©International Software Testing Qualifications Board

- BVAは状態遷移テストにおけるガード条件に使用できる。
- ドメインテストは、シナリオベースドテストにおいて、デシジョンテーブルやテスト対象システムで発生する変数から条件の値を決定するために使用することができる。
- シナリオベースドテストは、(ISTQB-TTAv4.0)で議論されているホワイトボックステスト技法であるデシジョンカバレッジと組み合わせることで、ビジネスプロセスにおける判定を厳密にカバーすることができる。例として、(Koomen et al., 2006)のプロセスサイクルテストを参照されたい。
- シナリオベースドテストをラウンドトリップカバレッジと組み合わせることで、周期的なビジネスプロセス活動特有のリスクに対応することができる。

3.5.2 テスト設計自動化のメリットとリスク

TAは、テスト技法(特にブラックボックステスト技法)を適用するためにツールを使用することがある。テスト設計を自動化する場合、TAはテストモデルを作成し、そのモデルからテストウェアを自動生成する。例えば、TAは状態遷移モデルを設計し、モデルベースドテストツールにラウンドトリップカバレッジのテストケースを生成させる。

テスト設計を自動化すると、テストの効率性と有効性が向上することが多い。その利点としては以下のものが挙げられる:

- **欠陥予防** テストのためのモデリングは、テストベースの品質を評価する効果的な方法である(5.2.1項参照)。
- **対応能力の拡張** 自動化により、組み合わせテスト、ランダムテスト、Nスイッチカバレッジなど、より複雑なテスト技法やカバレッジ基準の適用が可能になり、テストされていないコードのリスクが低減する。
- **わかりやすさの向上** ツールで指定されたテスト選択基準は、テスト条件をより明確かつ目に見える形で参照し、生成されたカバレッジの充足度合いをよりわかりやすく示す。
- **繰り返し作業の削減** テストモデルからテストウェアを生成できるため、テストの具体化といった手作業による繰り返し作業が減る。
- **メンテナンス工数の削減** テストモデルは、テストウェアを導出するために使用される信頼できる唯一の情報源である。その結果、テストモデルだけを保守すればよい。
- **テストウェアの欠陥削減** 手作業はエラーを起こしやすい。ツールによって生成されたテストウェアは品質と一貫性が高い。
- **チームコラボレーションの強化** ステークホルダーは、欠陥を見つけたり、テスト条件の理解を深めたりするために、テストモデルをレビューすることができる。
- **トレーサビリティの強化** テストモデルの要素をテスト条件にリンクすることは、テストケースそのものよりも簡単である。ツールが支援していれば、生成されたテストケースはそれらのリンクを継承し、テストにおける全体的なトレーサビリティを向上させる。
- **多様な出力形式** テストウェアは、他のツールやその後の活動の要件に応じて、さまざまな出力形式で生成することができる。

TAは、テスト設計を自動化するリスクも考慮しなければならない。これらのリスクには、モデルに示されていないテスト条件の見落とし、テストモデルのメンテナンス工数の過小評価、ステークホルダーがモデルを理解しにくいと感じること、テスト自動化の一般的なリスクなどが含まれる(ISTQB- CTFL, v4.0.1, 6.2節 参照)。

©International Software Testing Qualifications Board

4 品質特性のテスト - 60分

キーワード
適応性、互換性、柔軟性、機能適切性、機能完全性、機能正確性、機能適合性、機能テスト、インストール性、相互運用性、使用性、ユーザーエクスペリエンス

第4章の学習の目的

4.1 機能テスト
TA-4.1.1　(K2) 機能正確性、機能適切性、機能完全性テストを区別する。

4.2 使用性テスト
TA-4.2.1　(K2) テストアナリストが使用性テストにどのように貢献するか説明する。

4.3 柔軟性テスト
TA-4.3.1　(K2) テストアナリストが適応性テストおよびインストール性テストにどのように貢献するかを説明する。

4.4 互換性テスト
TA-4.4.1　(K2) テストアナリストが相互運用性テストにどのように貢献するかを説明する。

©International Software Testing Qualifications Board

品質特性のテストについて

本シラバスは、ISO 25010（ISO/IEC 25010, 2023）に規定されているソフトウェア品質モデルをガイドとして使い、TAとして注目すべき品質特性について述べている。ただし、使用性テストシラバス（ISTQB-UT、v1.0）と整合させるため、取り扱う使用性の用語は、この標準とは異なる。

付録Fでは、現行のISO 25010標準の品質モデルの概要、旧版と比較した変更点、および特定の品質特性のテストに焦点を当てた関連するISTQB®シラバスを示す。

4.1 機能テスト

機能テストは、TAのコアタスクの1つである。(ISTQB-CTFL、v4.0.1) では、テストタイプとしての機能テストを簡潔にまとめているが、本シラバスでは、機能適合性の副特性について、より詳細に論じている。

ISO 25010の製品品質モデル（ISO/IEC 25010, 2023）では、機能適合性の3つの副特性を区別している。これらの特性はすべて機能テストによって評価することができるが、すべての機能テスト活動がこれらに同じように対応するわけではない。TAは、機能適合性の特定の副特性に関連するプロダクトリスクに対処するために、適切なテストレベルとテスト技法を選択できるべきである。

機能完全性テストは、ソフトウェアの指定されたすべてのタスクと、想定されるユーザーの目的をカバーしなければならない。重要なのは、求められていることがすべて実装されているかどうかである。

機能完全性は、シーケンシャル開発モデルでは要件仕様のレビューによって、アジャイルソフトウェア開発ではユーザーストーリーの共同執筆時に受け入れ基準を含むユーザーストーリーについて議論することによって、できるだけ早い段階で対処すべきである。システムテスト、システム統合テスト、受け入れテストでは、機能完全性を動的にテストすることができる。

他のブラックボックステスト技法も適用可能であるが、シナリオベースドテストのような振る舞いに基づくテスト技法が適している。テストベース、テスト条件、テストケース間のトレーサビリティは、達成された機能完全性のレベルを決定する際に不可欠である。

機能正確性テストは、実際の結果が有効な入力に対しても無効な入力に対しても正しい（例えば、正確、精密、一貫性がある）かどうかという疑問に答えるものである。期待結果を詳細に提供する効果的なテストオラクルを発見することが極めて重要である。

機能正確性はどのテストレベルでもテストできる。シフトレフトの観点からは、ほとんどの機能正確性テストはコンポーネントテストとコンポーネント統合テストで行われるべきである。TAがこれらのテストレベルに責任を持たないとしても、TAはテスト目的を最良に達成するために貢献すべきである。

すべてのブラックボックステスト技法、経験ベースのテスト技法、コラボレーションベースのテストが適している。

機能適切性テストは、機能が指定されたタスクや目的の達成を容易にするものであることを検証する。実装されているものすべてがユーザーのニーズを満たしているかが焦点となる。

機能適切性テストには、特に対話型アプリケーションの場合、ユーザーインターフェース設計のレビューが含まれることがある。動的テストは、シーケンシャル開発モデルにおけるシステムテストと受け入れテストから始まる。アジャイルソフトウェア開発ではデモセッションから始める。

探索的テストやコラボレーションベースのテストが最も適している。さらに、振る舞いに基づくブラックボックステスト技法も適している。

©International Software Testing Qualifications Board

4.2 使用性テスト

使用性は、製品品質モデル(ISO/IEC 25010, 2023)におけるインタラクション容易性とISO 25019利用時の品質モデル(ISO/IEC 25019, 2023)における便益性を含む、ユーザーに関連する品質特性の広範な概念を指す。使用性テストに関する詳細事項は、(ISTQB-UT, v1.0)、(ISO 9241-210, 2019)、および(UXQB-FL, v4.01)を参照するとよい。

使用性テストは通常、次のような点を中心に評価する:

- インタラクション容易性 - ユーザーが特定の使用コンテキスト(*ISO/IEC 25010*, 2023)において、効果的、効率的、満足にタスクを完了できる。
- ユーザーエクスペリエンス - テスト対象とのやりとりの前、最中、およびその後のユーザーの知覚に対応する。
- アクセシビリティ - 障がいのあるユーザー、多様な文化的背景を持つユーザー、または言語の障壁を持つユーザーが、効果的かつ効率的にシステムを利用できる。

TAは、ターゲットユーザーグループ、彼らのゴール、彼らの使用時のコンテキスト、システムを使用する際の潜在的な困難、または否定的なユーザーエクスペリエンスに関する知識を活用することで、早い段階から使用性テストに協力することができる。

TAは、以下のように、主要な使用性評価技法に貢献することができる:

- **使用性レビュー**は、潜在的な使用性の問題や確立された基準からの逸脱を識別するために、使用性の専門家によって行われる。使用性レビューは、非形式的レビューからインスペクションまでさまざまである。TAは、レビュー基準を、ユーザーグループの特定のニーズ、特定のビジネス目標、優先度、および使用コンテキストに適応させることができる(一般的な使用性チェックリストを調整するなど)。
- **使用性テストセッション**では、将来のユーザーまたはその代表者が、あらかじめ定義されたタスクを実行する。このような関与により、タスクが効果的、効率的、および満足に完了できるかどうかを評価する。TAは、ペルソナ、ユーザーグループ、または運用プロファイルに従って、使用性テストセッションのシナリオ設計に貢献することができる。
- 評価とフィードバックを含む**ユーザーアンケートまたは調査**は、ユーザーの満足度を計測する。ユーザーアンケートの例としては、SUMI(Software Usability Measurement Inventory SUMI, 1991)およびWAMMI(Website Analysis and Measurement Inventory WAMMI, 1999)がある。TAは、使用コンテキストにおける対象ユーザーの特定のゴールに対応するために、アンケートの設計と回答の評価を支援することができる。

アクセシビリティテストにおいて、一般的なテスト目的は、標準への適合を検証することである。国際標準のウェブコンテンツのアクセシビリティガイドライン(WCAG, 2023)は、ウェブコンテンツのアクセシビリティの程度を示す3つの適合レベル(A, AA, AAA)を定義している。国内標準には、英国の平等法 (UK Government, 2010)および米国の米国人障害者法(U.S. Department of Justice, 2010)が含まれる。TAは、使用コンテキストを解析することで、求められる適合レベルと、意図するターゲットグループの具体的なニーズを識別することができる。

©International Software Testing Qualifications Board

4.3 柔軟性テスト

柔軟性テスト(移植性テストとも呼ばれる)は、テスト対象が使用コンテキストやシステム環境の変化に適応できることを検証する。ISO 25010の製品品質モデル(ISO/IEC 25010, 2023)では、柔軟性の副特性を以下のように区別している:

- 適応性
- 拡張性
- インストール性
- 置換性

柔軟性には、技術的側面と非技術的側面の両方がある。本節では、TAが適応性とインストール性のテストにどのように貢献できるかに焦点を当てる。拡張性と置換性は、技術的側面に関連し、それぞれ(ISTQB-PT、v1.0)および(ISTQB-TTA、v4.0)で論じている。

適応性テストは、テスト対象が意図された対象のハードウェア、ソフトウェア、その他の運用環境または使用環境に適応できること、または移行できることを検証する。

TAは、意図する対象環境(例えば、サポート対象のモバイルOSのバージョンや使用される可能性のあるブラウザーのバージョン)を識別し、これらの環境の組み合わせをカバーするテストを設計することで、適応性テストを支援する。これにはさまざまな環境パラメーターの構成を表すテストデータが必要となるため、組み合わせテストなどのテスト技法が適用されることが多い(*3.1.2*項参照)。もう1つの例は、対象環境間での互換性を確保するために、さまざまなプラットフォームのコンポーネントを顧客プロジェクトに統合することである。プロダクトリスクに応じて、TAはスモークテストまたはより包括的なテストスイートを設計、実行し、テスト対象が対象環境に正しく適応することを検証する。

適応性テストのよい実践例(例えば、早期に対象環境を定義する、組み合わせテストを使用する、新しい環境でスモークテストを行う、環境固有の欠陥をモニタリングする)に従うことで、TAは、ソフトウェアの寿命や使用性(例えば、異なる画面サイズでの使いやすさ)を制限する可能性のある欠陥を発見できる。テスト自動化エンジニアが実装する自動化されたクロスプラットフォームテストも、適応性テストにおけるTAの作業を支援するべきである。厳格な適応性テストは、環境固有の問題を早期に検出することができ、デプロイ後の頻繁なメジャーアップデートや再設計を回避し、メンテナンスコストを低減するのに役立つ。

インストール性テストは、テスト対象が指定した環境で正しくインストール、アンインストール、更新、再設定できることを検証する。インストール性テストは、単にインストール手順が完了まで動くかをチェックするだけではない。

TAの焦点となる典型的なインストール性テストの目的は以下の通りである:

- さまざまな環境パラメーター設定の下で、インストール手順が正しく実行されることを検証する。このため、適応性テストと同様に、組み合わせテストのようなテスト技法が役立つ。
- インストールまたはアップデート後にテスト対象が正しく動作するかどうかを判断するためのテストを設計し、実行する。
- ユーザーがソフトウェアのインストール、アンインストール、アップデートをどれだけ簡単に行えるかをチェックする。これには、インストール文書のレビューも含まれる。
- 特にモバイルアプリケーションの権限関連の動作をテストする(ISTQB-MAT, v1.0 を参照)。

4.4 互換性テスト

互換性テストは、テスト対象が使用されたときに、他のコンポーネントやシステムと互換性があるかどうかを検証するものである。ISO 25010製品品質モデル(*ISO/IEC 25010*, 2023)では、互換性の2つの副特性、相互運用性と

©International Software Testing Qualifications Board

共存性を区別している。したがって、互換性テストは、以下のテストタイプに細分化することができる:

- **相互運用性テスト**では、テスト対象が相互運用するコンポーネントやシステムとの互換性を検証する。これらのテストは通常ブラックボックスの機能テストであるためTAがその責任を負う。
- **共存性テスト**では、テスト対象が、他のコンポーネントやシステムと干渉することなく、対象環境を共有できることを検証する。この技術的なテストタイプは、テクニカルテストアナリストシラバス(ISTQB-TTA, v4.0)に説明がある。

相互運用性テストの目的は、2つ以上のコンポーネントまたはシステムが情報を交換でき、交換した情報を相互に利用できることを検証することである。情報交換がデータ変換を伴う場合、相互運用性テストはその変換の検証を含まなければならない。

相互運用性テストは、複数のシステムが連携し、データを共有し、共にタスクを実行する必要がある場合に特に重要である。これは特に、クラウドソリューション、ウェブサービス、マイクロサービス、コンテナ化、IoTといった最新のソフトウェアアーキテクチャーにおいて顕著である。

相互運用性は通常、さまざまなアーキテクチャーレベルで発生する。TAは、起こりうる相互作用を理解し、それらをカバーする適切なテスト条件を定義しなければならない。すべての相互作用が文書化されているとは限らない。TAは、アーキテクチャーや設計文書から、相互作用に関する情報を間接的に取得できる場合がある。したがって、相互作用のすべての重要な側面を確実にテストするために、この文書を理解することは極めて重要である。

相互運用性テストは、以下の欠陥を検出することができる:

- データ交換のためのデータ変換
- 交換されたデータの解釈または使用
- 通信フローとプロトコル
- 標準への準拠
- エンドツーエンドの機能性
- 設計文書

相互運用性テストは通常、統合テスト中に適用する。交換されたデータに焦点を当てるデータに基づくテスト技法、データの解釈や使用に関する振る舞いに基づくテスト技法、データ変換に関するエンドツーエンドの機能性やルールに基づくテスト技法などのブラックボックステスト技法は相互運用性テストに適している。経験ベースのテスト技法はブラックボックステストを有効に補完できる。機能テストや適応性テストのテストケースを相互運用性テストに再利用することもできる。

相互運用性に関する一般的な標準の例として、(*ISO 15745*, 2003)や(*ISO 16100*, 2009)がある。*ETSI (ETSI EG 202 237 v1.2.1*, 2010)は、電気通信分野における具体的な相互運用性テスト方法論を例示している。

©International Software Testing Qualifications Board

5 ソフトウェア欠陥予防 - 225分

キーワード

アドホックレビュー、チェックリストベースドレビュー、欠陥予防、モデルベースドテスト、パースペクティブベースド リーディング、レビュー技法、ロールベースドレビュー、根本原因分析、シナリオベースドレビュー、テスト結果

第5章の学習の目的

5.1 欠陥予防の実践

TA-5.1.1 (K2)テストアナリストが欠陥予防にどのように貢献できるかを説明する。

5.2 フェーズ内封じ込めの支援

TA-5.2.1 (K3)仕様書の欠陥を検出するためにテスト対象のモデルを使用する 。

TA-5.2.2 (K3)欠陥を発見するためにレビュー技法をテストベースに適用する。

5.3 類似欠陥の再発の軽減

TA-5.3.1 (K4)欠陥検出に関する改善の可能性を識別するためにテスト結果を分析する。

TA-5.3.2 (K2)欠陥分類が根本原因分析にどのように役立つかを説明する。

©International Software Testing Qualifications Board

ソフトウェア欠陥予防について

欠陥予防のゴールは、作業成果物に欠陥が(再)発生する可能性を低減し、SDLCの後続フェーズへの欠陥の流出を緩和するアクションを実施することである。これらの努力は、コストと労力の削減、生産性の向上、プロダクト品質の改善など、さまざまな重要な利益をもたらす。欠陥予防は、チーム全体の責任である。TAは、特定の知識と経験を活用して、それに貢献することができる。

欠陥予防の実践には以下のようなものがある:

- 品質保証活動の一環として、欠陥の混入を防ぐ。
- 欠陥がSDLCの後続フェーズに見逃されるのを防ぐ(5.2節を参照)。
- 類似欠陥の再発を防ぐ(5.3節参照)。

5.1 欠陥予防の実践

TAは、ドメインの知識、テストの専門知識、分析スキルを駆使して、いくつかの方法で欠陥予防に貢献することができる。例えば、以下のようなものがある:

- リスク分析への参加 - 識別したリスクが適切に軽減されるようにする(例えば、最も適切なテスト技法を選択する)。
- 要件、モデル、仕様書のレビュー - テストベースに存在する欠陥の早期発見を可能にすることでコードへの流出を防ぎ、結果としてその修正コストを大幅に削減する。
- ふりかえりへの参加 - テスト分析、テスト設計、テスト実装、およびテスト実行における改善の可能性(例えば、より効果的なテスト技法の使用、特定のリスク領域のテストに的を絞る、偽陽性結果および偽陰性結果の両方を低減するためのテストデータおよびテスト環境の改善など)を識別し、欠陥の流出をより確実に予防する。
- 欠陥データの収集と評価 - 欠陥に関する詳細なデータを収集することで、欠陥の分類と統計分析を可能にし、根本原因分析とプロセス改善を支援する。
- 根本原因分析に参加し、識別した根本原因に対処するための是正措置を提案し、欠陥の再発を予防する。

欠陥予防活動に参加するだけでなく、TAは、提案した手段が望ましい効果をもたらしたかを(通常はテストマネージャーと相談しながら)評価する。これらの手段の有効性評価に役立つメトリクスの例には、次のようなものがある:

- 欠陥除去効率(DRE) - リリース前に除去された欠陥と欠陥の総数の比率を測定する。(未知の)分母は、ある時点まで(例えば、リリース後6ヶ月まで)に発見された欠陥の総数で推定または置き換えることができる。DREが高いということは、本番環境に見逃された欠陥が少ないということであり、これは欠陥予防の実践が優れていることを意味することがある。しかし、DREは、予防と検出によって発見された欠陥を区別しない。
- フェーズ内封じ込め有効性(PCE) - そのフェーズで作り込まれた欠陥の総数に対する、同じフェーズ内で作り込まれ、かつ除去された欠陥の割合を測定する。PCEが高いほど、後のフェーズに見逃された欠陥が少ないことを示す。
- 品質コスト - 欠陥予防、欠陥検出、除去コストの関係を示す(ISTQB-TM、v3.0、3.2.1項を参照)。

©International Software Testing Qualifications Board

5.2 フェーズ内封じ込めの支援

フェーズ内封じ込めの目的は、欠陥が作り込まれたのと同じSDLCフェーズ内で欠陥を検出し、除去することである。アジャイルソフトウェア開発では、シフトレフトアプローチを同様に使うことができる。

この方針は、品質コストを削減する。テストベースはテスト分析とテスト設計への重要なインプットであるため、TAはテストベースの品質を評価することにより、フェーズ内封じ込めに最も貢献することができる。

テストベースの品質に早期に注意を払うことで、後の労力を最小限に抑え、欠陥が後続フェーズに流出するのを防ぐことができる。本シラバスでは、TAがテストベースの欠陥を発見するための2つの一般的な方法であるテストの意図のモデリングと、さまざまなレビュー技法(*ISO/IEC 20246*, 2017)を用いたテストベースのレビューについて説明する。

5.2.1 欠陥検出でのモデルの利用

モデリングは、システムの抽象化を一定の精度と詳細さで提供する。これは、ステークホルダーが開発中のシステムをよりよく理解するのに役立つ。後述するように、モデリングは少なくとも3つの方法でフェーズ内封じ込めを支援することができる:

- 仕様書の欠陥を検出する。
- モデルの欠陥を検出する。
- モデルベースドテストを使用してテスト対象の欠陥を検出する。

仕様書の欠陥を検出する。仕様書は多くの場合、非形式的に書かれたテキストとして提供される。TAは、これらの仕様に対してモデルを使って形式的に表現できる。モデルを作成する際、テスト条件(要件など)はモデル要素にマッピングし、トレーサビリティのためにモデル要素にリンクする。モデル内のテスト条件を形式化し視覚化することで、不完全性、不整合、曖昧さなどの欠陥を効率的に明らかにできる。レビューが仕様を当初の形式のままチェックするのに対して、モデリングの強みは仕様が変換されることである。その結果、TAは検出された欠陥に対する適切な解決策の発見にも貢献する。

データに基づくモデルには、ドメインモデルと組み合わせテストモデルがある。これらは、重複するパーティション、ドメインカバレッジのギャップ、空のパーティション、パラメーターの欠落や不正確な組み合わせなどのドメインの欠陥をTAが検出することを可能にする。

振る舞いに基づくモデルには、CRUDマトリクス、状態ベースモデル、シナリオモデルなどがある。これらのモデルにより、TAは、不完全または一貫性のないエンティティのライフサイクル、欠落または欠陥のある状態遷移、デッドロック、無限ループ、曖昧または一貫性のないシステム動作、または欠落した例外処理を検出することができる。

デシジョンテーブルやメタモルフィック関係のようなルールに基づくモデルは、TAがビジネスルールの欠陥(抜け、不整合、曖昧さ、冗長性、エラーを起こしやすいシナリオ、複雑なビジネスロジックなど)を検出することを可能にする。

モデリングは、命名、データ値(境界値など)、入力または出力の不整合などの欠陥も検出できる。また、欠落している情報、不完全な情報、曖昧な情報、不要な情報を特定することもできる。

モデルの欠陥を検出する。テストベースにモデルが含まれている場合、TAはこれらのモデルを解析して欠陥を検出することができる。本シラバスでは、ソフトウェアテストで使用される3つの典型的なモデル、すなわち状態遷移図(*3.2.2項*参照)、アクティビティ図(*3.2.3項*参照)、デシジョンテーブル(*3.3.1項*参照)における欠陥検出について

©International Software Testing Qualifications Board

説明する。上記のモデルにおける欠陥の例には、以下のようなものがある:

- 状態遷移図 - 状態の欠落/誤り、不適切な遷移、誤ったガード条件やアクション、冗長な状態や到達不可能な状態、非決定的な動作。
- アクティビティ図 - アクションの欠落、到達不能、または行き止まり、アクションの順序の誤り、判定ノードにおける誤った、排他的でない、あるいは不完全なガード条件、意図しない動作につながる同期ポイントの欠落あるいは、並列フローの不適切な同期。
- デシジョンテーブル - 重複した、不整合な、あるいは実現不可能なルール、不完全性(条件の組み合わせの欠落や、与えられた条件の組み合わせに対するアクションの欠落など)。

すべてのモデルには、構文エラー、タイプミス、重複、モデル要素の命名の不統一などの欠陥も含まれる可能性がある。

モデルベースドテスト(MBT)を用いて欠陥を検出する。このテストアプローチでは、MBTツールが、TAによって定義されたMBTモデルとテスト選択基準に基づいて、テストケースやその他のテスト設計におけるテストウェアを設計し、そして生成する。MBTは、MBTモデルに従ってテスト対象の期待される動作を包括的にカバーし、体系的に探索することができるため、仕様書の不正の発見に効果的である。MBTモデル、特にグラフィカルなモデルは、ステークホルダーとのコミュニケーションを促進し、テストベースに対する共通の認識と理解を生み出す。MBTモデルの詳細については、モデルベースドテスト担当者シラバス(ISTQB-MBT,v1.1)を参照のこと。

5.2.2 レビュー技法の応用

十分に定義されたテストベースは、作業成果物の品質を確保し、プロジェクトを成功に導くための礎となる。テストベースのレビューは、欠陥の早期発見と対処に役立ち、欠陥が後続フェーズに見逃されるのを防ぐ。

TAは、テストベースの欠陥を識別するために、個別レビューにおいてさまざまなレビュー技法を用いることができる。最も適切なレビュー技法を選択することで、レビューの効率性と有効性を高めることができる。この選択は、レビューのゴール、プロジェクト目的、利用可能なリソース、テストベースのタイプ、関連リスク、ビジネスドメイン、企業文化などの要因を考慮する必要がある。

以下、本シラバスでは、TAがよく使う5つのレビュー技法について述べる。

アドホックレビューは、構造化されたプロセスなしに、レビューアによって非形式的に行われる。レビューアには、どのように作業を行うべきかについてのガイダンスはほとんど、あるいはまったく提供されない。アドホックレビューにはほとんど準備は必要なく、レビューアのスキルに大きく依存する。レビュー中、レビューアはテストベースを読み、不正を発見するたびにそれを文書化する。この手法をマネジメントせずに放置すると、複数のレビューアによる重複した不正の報告が大量に発生する可能性がある。

チェックリストベースドレビューでは、あらかじめ定義されたチェックリストに照らしてテストベースを評価する。チェックリストは、レビューアに特定の確認事項をチェックするよう注意を促し、レビューを客観化することができる。チェックリストは一般的なものから、品質特性、テスト目的、テストベースのタイプに特化したものまである。TAは、チェックリストをテストベースのタイプ、リスクレベル、またはテスト条件に合わせて調整する。これにより、レビューがテストベースの最も重要な側面に焦点を当てることができる。チェックリストは、過去に見過ごされた欠陥を用いて定期的に更新するべきである。チェックリストを最新の状態に保つことで、新たに確認された不正の見落としを防ぐことができる。チェックリストはすべてを網羅するものではない。したがって、TAが行うチェックは、リストアップされた項目に限定されない。これによって欠陥の検出が最大限になり、チェックリストでは明確にカバーできない不正をTAが捉えることができる。

シナリオベースドレビューでは、プロセスや活動をシミュレーションして、不正を識別し、テストベースを洗練する。こ

©International Software Testing Qualifications Board

のレビュー技法は、テストベースがユースケースやアクティビティ図のようなシナリオベースの形式である場合に最も効果的である。このような場合、レビューアは、作業成果物の予想される使用方法に基づいて「ドライラン」を行うことができる。シナリオは貴重なガイドラインを提供するが、レビューアは文書化されたシナリオに制約されることはなく、より多くの不正を識別するためにシナリオを超えて考えることも必要である。

ロールベースドレビューでは、レビューアに特定の役割や責任を割り当てる。典型的な役割は、特定のエンドユーザーのタイプ（例：経験者、未経験者、シニア、子供ユーザー）、または組織内の特定の役割（例：システム管理者、一般ユーザー）に基づいている。各役割は、ペルソナ（すなわち、特定のユーザーグループの特性、ニーズ、目標、嗜好を表すように設計された、具体的だが架空のキャラクター）によって記述することができる。ロールベースドレビューでは、レビューアがそれぞれの役割に基づいて責任を分担することで、各自がテストベースの特定の側面に集中し、包括的なカバレッジを確保すると同時に、不正の重複を避けることができる。

パースペクティブベースドリーディングでは、さまざまな視点や観点（設計者、テスト担当者、マーケティング担当者、システム管理者、エンドユーザーなど）からテストベースをレビューする。これにより、個々のレビューがより深くなり、かつレビューア間で発見した不正の重複が少なくなる。加えて、パースペクティブベースドリーディングでは、レビューアに対して、レビュー対象のテストベースを使って、そこから導出するはずの作業成果物を生成しようとすることが求められる。例えば、テスト担当者は、要件仕様に基づいて受け入れテストの草案を作成し、必要な情報がすべて含まれているかどうかを確認する。

5.3 類似欠陥の再発の軽減

TAは、ソフトウェアへの欠陥の再発を最小限に抑えるために、主体的に貢献ができる。本シラバスでは、欠陥の再発を軽減するための2つのアプローチ、すなわちテスト分析およびテスト設計を改善するためのテスト結果の分析と、欠陥分類による根本原因分析の支援について説明する。

5.3.1 欠陥検出を改善するためのテスト結果の分析

テスト結果は、TAが故障を識別することを可能にするだけでなく、欠陥検出の有効性を向上させるためのフィードバックをTAに提供する。テスト結果を解析するために一般的に使用される技法を以下に説明する。

予測した欠陥クラスターと実際の欠陥クラスターの分析　通常、少数のコンポーネントに大部分の欠陥が含まれている（ISTQB-CTFL、v4.0.1、1.3節参照）。テスト後、TAは欠陥の発生しやすい領域を識別し、予測した欠陥クラスターと実際の欠陥クラスターを比較することができる。不一致があった場合、予想よりも多くの欠陥が発見された領域に対して、より入念なテストをすることができる。クラスターを決定する際には、明確性と一貫性を確保するために測定可能な基準を用いる必要がある（欠陥密度や欠陥の重要度など）。これらの基準の中で、欠陥の重要度が重要な役割を果たすべきである。致命的欠陥や重大欠陥の小さなクラスターは、通常、軽微な欠陥や外観上の欠陥の大きなクラスターよりも重要である（すなわち、より厳密なテストが必要である）。

欠陥検出率（DDP）分析　DDPは、各テストレベルにとって最も重要なテスト有効性の尺度のひとつである。DDPを計算する場合、流出した欠陥の数は、対象としているテストレベルで検出可能だった欠陥に限定すべきである。一貫性を確保するために、時間的制限（例えば、リリース後の定義された期間内に発見された欠陥）や除外基準（例えば、サードパーティコンポーネントや特定の顧客環境の欠陥）のような、欠陥を数える明確な境界を設定すべきである。あるテストレベルにおいてDDPが低いということは、流出した欠陥の割合が高く、欠陥検出が効果的でないことを意味する。このような場合、TAはその理由を分析し、そのテストレベルをより的を射た入念なものにするための改善案を提案すべきである。通常、見逃された欠陥の削減の優先度は、その重要度によって異なるため、DDPは重要度別に分けるのが最善である。

構造カバレッジ分析では、テスト対象の特定の領域をテストがどの程度まで実施したかを評価する。カバレッジの

©International Software Testing Qualifications Board

低い領域を識別することで、TAはその領域に的を絞ってテストを行うことができる。カバレッジを高めることで、これまで見逃されていた新たな欠陥の発見に役立つ。ステートメントカバレッジ、ブランチカバレッジ、ニューロンカバレッジのような構造カバレッジは、通常テストツールで測定する。追加でカバーする領域を選択する際には、そのリスクレベルを考慮する必要がある。

テストギャップ分析では、テストが最近のコード変更をどの程度実施しているかを評価する。これによってTAは、カバレッジの低いすべての領域（例えば、長い間変更されておらず、以前のリリースのためにテストが実施されたコード）を対象とするのではなく、特にエラーが発生しやすい領域（すなわち、まったくテストされていない新規の変更）に、追加のテスト工数を集中させることができる。

欠陥発生パターン分析　プロジェクトの連続するフェーズ（例：イテレーション）において発見される欠陥の数や密度は、時間の経過に伴うこれらの値の理論的な分布を示すパターンと比較することができる。欠陥の発生パターンの古典的な例は、*レイリー*モデルである（Elsayed, 2021）。これは、単一のピークを持ち、右側に偏っており、発見される欠陥の期待数が時間とともに最初に増加し、最大値に達した後、ゼロに向かってゆっくりと減少することを示している。このようなパターンの分析に基づき、既存のテストケースの有効性と改善の可能性を推測することができる。例えば、パターン上では欠陥検出数が増加するはずの時期に、低いレベルで一定のままであると仮定する。その場合、それは既存のテストの有効性が低すぎて、それ以上の欠陥を検出できていないことを意味している可能性がある。

上述した分析手法では、欠陥数やDDPなど、さまざまな欠陥関連のメトリクスを使用する。これらのメトリクスは、テスト結果（合格/不合格テスト数など）、欠陥レポート、構造メトリクス（コードカバレッジや欠陥密度など）に基づいて算出する。ただし、これらのメトリクスは、テスト結果から見かけほど簡単に読み取れるとは限らないことに注意する必要がある。例えば、不合格となったテストの数は、テストによって検出された欠陥の数と必ずしも同じではない。テスト結果と欠陥の関係は多対多になり得るため、実際に検出した欠陥の数を計算するには、デバッグプロセスの結果を注意深く分析しなければならない。複数のテストが同じ欠陥を検出することもあれば、1つのテストが複数の欠陥を検出することもある。さらに、欠陥の重要度はテストの重要度とは異なる場合がある。重要なテストケースが外観上の欠陥で不合格となることもあるからである。

5.3.2 欠陥分類による根本原因分析の支援

根本原因分析（RCA）とは、欠陥の症状だけでなく、その背後にある根本原因を識別し、それに対処するための技法である。RCAは、品質改善のための構造化されたアプローチを支援するものであり、その一番の目的は欠陥の再発を予防することである。TAは、欠陥や故障の根本原因を識別するためにさまざまな技法を用いる（欠陥分類法、なぜなぜ分析、特性要因図、パレート分析など）。

古典的なRCAでは、欠陥を解決した後、その欠陥の専門家がかなり詳細に調査する。しかし、通常、分析が必要な欠陥は数多く存在する。したがって、欠陥ごとに予防処置計画を立てることは、非常に非効率的で時間がかかる。この問題にアプローチする1つの方法は、欠陥を分類し、発生した欠陥の種類に対してRCAを実施することである。

欠陥分類は、個々の欠陥が、開発プロセスやテスト対象システムに関する多くの情報を含んでいるという認識に基づいている。欠陥分類により、TAは欠陥から開発プロセスのさまざまな側面に関する情報を抽出し、それをプロセス測定指標に変換することができる。これにより、開発中にどのようなエラーが発生したかという洞察が得られ、プロセス改善に役立つ。欠陥の分類は、定量的な欠陥統計と定性的なRCAの間のギャップを埋める。RCAを効果的に支援するために、欠陥は、初期のテストから本番環境に至るまで、SDLC全体を通して統一的に分類されるべきである。

TAは、ソフトウェア欠陥の分類を標準化するために、所属組織を支援すべきである。これにより、開発担当者や組織間のコミュニケーションや欠陥に関する情報交換が改善し、RCAが容易になる。

欠陥分類法の例としては、以下のようなものがある：

©International Software Testing Qualifications Board

- 直交欠陥分類(ODC)(Chillarege, 1992)は、各欠陥を直交する(すなわち、相互に排他的な)属性に分類するもので、欠陥がレポートされたときと欠陥が修正されたときの両方で収集する。
- IEEE 1044 (IEEE 1044, 2009)は、ソフトウェア不正を分類するための標準であり、故障と欠陥の分類のためのコア属性を提供する。
- 重要度ベースの分類:欠陥の重要度(critical, major, minor, trivial など)に基づいて分類する。
- 欠陥分類モデル:(Beizer,1990)や(Catolino et al,2019)などがある。

欠陥は、(*ISO/IEC 25010*, 2023)やFURPSモデル(Grady et al, 1987)などのソフトウェア品質モデルを使用して、品質特性にマッピングすることもできる。

RCAに関する詳細情報は、(ISTQB-ITP, v1.0)を参照のこと。

©International Software Testing Qualifications Board

6 参考文献

標準

ETSI EG 202 237 v1.2.1: Internet Protocol Testing (IPT), 2010. Standard. European Telecommunications Standards Institute.

IEEE 1044: Standard Classification for Software Anomalies, 2009. Standard. Institute of Electrical and Electronics Engineers.

ISO 15745: Industrial automation systems and integration– Open systems application integration framework, 2003. Standard. International Organization for Standardization.

ISO 16100: Industrial automation systems and integration– Manufacturing software capability profiling for interoperability, 2009. Standard. International Organization for Standardization.

ISO 26262: Road vehicles– functional safety– Part 6: Product development at the software level, 2018. Standard. International Organization for Standardization.

ISO 9241-210: Ergonomics of human-system interaction, part 210: Human-centred design for interactive systems, 2019. Standard. International Organization for Standardization.

ISO/IEC 20246: Software and systems engineering– Work product reviews, 2017. Standard. International Organization for Standardization.

ISO/IEC 25010: Systems and software Quality Requirements and Evaluation (SQuaRE)– Product quality model, 2023. Standard. International Organization for Standardization. 「製品品質モデルJIS X 25010：2025（ISO/IEC 25010：2025）」

ISO/IEC 25019: Systems and software Quality Requirements and Evaluation (SQuaRE)– Quality-in-use model, 2023. Standard. International Organization for Standardization.

ISO/IEC/IEEE 29119-3: Software testing, Part 3: Test documentation, 2021. Standard. International Organization for Standardization.

ISO/IEC/IEEE 29119-4: Software testing, Part 4: Test techniques, 2021. Standard. International Organization for Standardization.

ISO/IEC/IEEE 29119-5: Software testing, Part 5: Keyword-Driven Testing, 2016. Standard. International Organization for Standardization.

OMG® BPMN:Business Process Model and Notation, version 2.0, 2013. Standard. Object Management Group, OMG®.

OMG® DMN:Decision Model and Notation, version 1.5, 2024. 2024-01. Standard. Object Management Group.

OMG® UML:Unified Modeling Language, version 2.5, 2017. Standard. Object Management Group.

RTCA DO-178C: Software Considerations in Airborne Systems and Equipment Certification, 2011. Standard. Radio Technical Commission for Aeronautics, RTCA Inc.

ISTQB®ドキュメント

ISTQB-ACT, ISTQB®, 2019. Certified Tester Specialist Mobile Acceptance Testing: Syllabus. Version 1.0.

©International Software Testing Qualifications Board

ISTQB-AI, ISTQB®, 2021. Certified Tester AI Testing: Syllabus. Version 1.0.

ISTQB-CTFL, ISTQB®, 2024. Certified Tester Foundation Level: Syllabus. Version 4.0.1.

ISTQB-ITP,ISTQB®, 2011. Certified Tester Expert Level Improving the Test Process: Syllabus. Version 1.0.

ISTQB-MAT, ISTQB®, 2019. Certified Tester Specialist Mobile Application Testing: Syllabus. Version 1.0.

ISTQB-MBT, ISTQB®, 2024. Certified Tester Model-Based Tester: Syllabus. Version 1.1.

ISTQB-PT, ISTQB®, 2018. Certified Tester Performance Testing: Syllabus. Version 1.0.

ISTQB-TAE, ISTQB®, 2024. Certified Tester Test Automation Engineering: Syllabus. Version 2.0.

ISTQB-TM, ISTQB®, 2024. Certified Tester Advanced Level Test Management: Syllabus. Version 3.0.

ISTQB-TTA, ISTQB®, 2021. Certified Tester Advanced Level Technical Test Analyst: Syllabus. Version 4.0.

ISTQB-UT, ISTQB®, 2018. Certified Tester Usability Testing: Syllabus. Version 1.0.

書籍

AMMANN, Paul; OFFUTT, Jeff, 2008. Introduction to Software Testing. Cambridge University Press.

BEIZER, Boris, 1990. Software Testing Techniques (2nd Ed.) International Thomson Computer Press. 「ソフトウェアテスト技法,日経BP」

BINDER, Robert V., 2000. Testing Object-Oriented Systems. Addison-Wesley.

ELSAYED, Elsayed A.,2021. Reliability Engineering. Wiley.

FORGÁCS, Istvan; KOVÁCS, Attila, 2019. Practical Test Design: Selection of traditional and automated test design techniques. BCS, The Chartered Institute for IT.

FORGÁCS, Istvan; KOVÁCS, Attila, 2024. Modern Software Testing Techniques. Apress.

GRADY, Robert; CASWELL, Deborah, 1987. Software Metrics: Establishing a Company-wide Program. Prentice Hall.

HENDRICKSON, Elisabeth, 2013. Explore It! Pragmatic Bookshelf.

JORGENSEN, Paul, 2014. Software Testing. A Craftsman’s Approach. CRC Press. KANER, Cem; FALK, Jack; NGUYEN, Hung Q., 1999. Testing Computer Software. Wiley.

KANER, Cem; PADMANABHAN, Sowmya; HOFFMAN, Douglas, 2013. The Domain Testing Workbook. Context Driven Press.

KOOMEN, Tim; AALST, Leo van der; BROEKMAN, Bart; VROON, Michiel, 2006. TMap Next for resultdriven testing.UTN Publishers.

KUHN, D. Richard; YU, Lei; KACKER, Raghu N., 2013. Introduction to Combinatorial Testing. CRC Press.

記事

©International Software Testing Qualifications Board

ALYAHYA, Sultan, 2020. Crowdsourced Software Testing: A Systematic Literature Review. Information and Software Technology. Vol. 127, p. 106363. Available from doi: 10.1016/j.infsof.2020.106363.

ANTONIOL, Giuliano; BRIAND, Lionel; DI PENTA, Massimiliano; LABICHE, Yvan, 2002. A case study using the round-trip strategy for state-based class testing. Proceedings 13th International Symposium on Software Reliability Engineering, pp. 269–279. isbn 0-7695-1763-3. Available from doi: 10.1109/ISSRE.2002.1173268.

ARCURI, Andrea; IQBAL, Muhammad Zohaib; BRIAND, Lionel, 2012. Random Testing: Theoretical Results and Practical Implications. IEEE Transactions on Software Engineering. Vol. 38, pp. 258–277. Available from doi: 10.1109/TSE.2011.121.

BARR, Earl; HARMAN, Mark; MCMINN, Phil; SHAHBAZ, Muzammil; YOO, Shin, 2014. The Oracle Problem in Software Testing: A Survey. IEEE Transactions on Software Engineering. Vol. 41, no. 5, pp. 507–525. Available from doi: 10.1109/TSE.2014.2372785.

BOCHMANN, Gregor; PETRENKO, Alexandre, 1994. Protocol Testing: Review of Methods and Relevance for Software Testing. ISSTA '94: Proceedings of the 1994 ACM SIGSOFT international symposium on Software testing and analysis, pp. 109–124. Available from doi: 10.1145/186258.187153.

CATOLINO, Gemma; PALOMBA, Fabio; ZAIDMAN, Andy; FERRUCCI, Filomena, 2019. Not All Bugs Are the Same: Understanding, Characterizing, and Classifying Bug Types. Journal of Systems and Software. Vol. 152, pp. 165–181. Available from doi: 10.1016/j.jss.2019.03.002.

CHILLAREGE, R. et al., 1992. Orthogonal Defect Classification- A Concept for In-Process Measurements. IEEE Transactions on Software Engineering. Vol. 18, pp. 943–956. Available from doi: 10.1109/32. 177364.

CHOW,Tsun, 1978. Testing Software Design Modeled by Finite-State Machines. IEEE Transactions on Software Engineering. Vol. 4, pp. 178–187. Available from doi: 10.1109/TSE.1978.231496.

COHEN, D.M.; DALAL, Siddhartha; KAJLA, A.; PATTON, G.C., 1994. The Automatic Efficient Test Generator (AETG) system. Proceedings of 1994 IEEE International Symposium on Software Reliability Engineering, pp. 303–309. isbn 0-8186-6665-X. Available from doi: 10.1109/ISSRE.1994.341392.

ENGSTRÖM, Emelie; RUNESON, Per; SKOGLUND, Mats, 2010. A systematic review on regression test selection techniques. Information and Software Technology. Vol. 52, pp. 14–30. Available from doi: 10.1016/j.infsof.2009.07.001.

GHAZI, Ahmad Nauman; GARIGAPATI, Ratna; PETERSEN, Kai, 2017. Checklists to Support Test Charter Design in Exploratory Testing. Agile Processes in Software Engineering and Extreme Programming. XP 2017. isbn 978-3-319-57632-9. Available from doi: 10.1007/978-3-319-57633-6_17.

HAREL, David, 1987. Statecharts: A Visual Formalism For Complex Systems. Science of Computer Programming. Vol. 8, pp. 231–274. Available from doi: 10.1016/0167-6423(87)90035-9.

HUANG, Rubing; SUN, Weifeng; XU, Yinyin; CHEN, Haibo; TOWEY, Dave; XIA, Xin, 2019. A Survey on Adaptive Random Testing. IEEE Transactions on Software Engineering. Vol. 47, no. 10, pp. 2052–2083. Available from doi: 10.1109/TSE.2019.2942921.

JENG, Bingchiang; WEYUKER, Elaine, 1994. A Simplified Domain-Testing Strategy. ACM Transactions on Software Engineering and Methodology. Vol. 3, pp. 254–270. Available from doi: 10.1145/196092. 193171.

JUERGENS, Elmar; PAGANO, Dennis; GOEB, Andreas, 2018. Test Impact Analysis: Detecting Errors Early Despite Large, Long-Running Test Suites. Whitepaper, CQSE GmbH.

KUHN, D.; WALLACE, Dolores; JR, A.M., 2004. Software Fault Interactions and Implications for Software Testing. IEEE Transactions on Software Engineering. Vol. 30, pp. 418–421. Available from doi: 10.1109/TSE.2004.24.

LEICHT, Niklas; BLOHM, Ivo; LEIMEISTER, Jan Marco, 2017. Leveraging the Power of the Crowd for

©International Software Testing Qualifications Board

Software Testing. IEEE Software. Vol. 34, pp. 62–69. Available from doi: 10.1109/MS.2017.37.

OFFUTT, A. Jefferson, 1992. Investigations of the software testing coupling effect. ACM Transactions on Software Engineering and Methodology. Vol. 1, pp. 5–20.

RECHTBERGER, Vaclav; BURES, Miroslav; AHMED, Bestoun, 2022. Overview of Test Coverage Criteria for Test Case Generation from Finite State Machines Modelled as Directed Graphs. IEEE International Conference on Software Testing, Verification and Validation Workshops (ICSTW), Valencia, Spain, pp. 207–214.

RWEMALIKA, Renaud; KINTIS, Marinos; PAPADAKIS, Mike; LE TRAON, Yves; LORRACH, Pierre, 2019. On the Evolution of Keyword-Driven Test Suites. 12th IEEE Conference on Software Testing, Validation and Verification (ICST), pp. 335–345.

SEGURA, Sergio; FRASER, Gordon; SANCHEZ, Ana; RUIZ-CORTÉS, Antonio, 2016. A Survey on Metamorphic Testing. IEEE Transactions on Software Engineering, pp. 46–53.

SEGURA, Sergio; TOWEY, Dave; ZHOU, Zhi Quan; CHEN, Tsong Yueh, 2020. Metamorphic Testing: Testing the Untestable. IEEE Software. Vol. 42, no. 9, pp. 805–824.

WU, Huayao; NIE, Changhai; PETKE, Justyna; JIA, Yue; HARMAN, Mark, 2020. An Empirical Comparison of Combinatorial Testing, Random Testing and Adaptive Random Testing. IEEE Transactions on Software Engineering. Vol. 46, no. 3, pp. 302–320.

Webページ

COMMISSION, European, 2016. General Data Protection Regulation: Regulation (EU) 2016/679 [online]. [visited on 2024-09-15]. Available from: https://commission.europa.eu/law/law-topic/data-protection/legal-framework-eu-data-protection_en?.

COMPUTERSOCIETY, IEEE; JTC 1/SC7, ISO/IEC, 2024. SE VOCAB: Software and Systems Engineering Vocabulary [online]. [visited on 2024-09-15]. Available from: https://pascal.computer.org/.

CZERWONKA, Jacek, 2004. Pairwise Testing: Combinatorial Test Case Generation [online]. [visited on 2024-09-15]. Available from: www.pairwise.org.

HEALTH, U.S. Department of; SERVICES, Human, 2024. Health Insurance Portability and Accountability Act: Standards for Privacy of Individually Identifiable Health Information (Privacy Rules) [online]. [visited on 2024-09-15]. Available from: https://www.hhs.gov/hipaa/for-professionals/privacy/laws-regulations/index.html. IREB-GLOSSARY, IREB®, 2024.

The CPRE Glossary: Core terms of Requirements Engineering [online]. [visited on 2024-09-15]. Available from: https://glossary.istqb.org.

ISTQB-GLOSSARY, ISTQB®, 2024. Standard Glossary of Terms Used in Software Testing [online]. [visited on 2024-09-15]. Available from: https://glossary.istqb.org.

Site of Software Test Design, 2020 [online]. [visited on 2024-09-15]. Available from: https://test-design.org/.

SUMI, 1991. Software Usability Measurement Inventory: Standard evaluation questionnaire for assessing quality of use [online]. [visited on 2024-09-15]. Available from: sumi.uxp.ie.

U.S. DEPARTMENT OF JUSTICE, Civil Rights Division, 2010. Americans with Disabilities Act: Guidance & Resource Materials [online]. [visited on 2024-09-15]. Available from: www.ada.gov/resources/.

UKGOVERNMENT,Equalities Office, 2010. Equality Act 2010: guidance: Information and guidance on the Equality Act 2010, including age discrimination and public sector Equality Duty. [online]. [visited on 2024-09-15]. Available from: www.gov.uk/guidance/equality-act-2010-guidance.

©International Software Testing Qualifications Board

WAMMI, 1999. Website Analysis and Measurement Inventory: Professional service for analyzing user experience [online]. [visited on 2024-09-15]. Available from: www.wammi.com.

WCAG, 2023. Web Content Accessibility Guidelines 2.2: W3C Recommendation [online]. [visited on 202409-15]. Available from: www.w3.org/TR/WCAG22/.

これまでの参考文献は、インターネットやその他の場所で入手可能な情報を指している。これらの参考文献は、本シラバスの発行時に確認されたものであるが、ISTQB® は、その参考文献が利用できなくなった場合、責任を負うことはできない。

©International Software Testing Qualifications Board

7 付録A - 学習の目的／認知レベル

本シラバスの具体的な学習の目的は、各章の冒頭に示されている。シラバスの各トピックは、その学習の目的に沿って検討される。

学習の目的は、以下のように、知識の認知レベルに対応する動作動詞で始まる。

レベル1：想起 （K1）
用語や概念を記憶し、認識し、想起する。

動作動詞：想起する、認識する。
注：上級レベルのシラバスには、具体的なK1レベルの学習の目的はない。ただし、第1章から第5章までのシラバスの内容、および各章の見出しのすぐ下にキーワードとして記載されている用語の定義は、学習の目的に明示されていなくても、記憶（K1）しなければならない。

レベル2：理解する （K2）
トピックに関連したステートメントの理由や説明を選択し、テストコンセプトについて要約、比較、分類、例を挙げることができる。

動作動詞：分類する、比較する、区別する、説明する、例を挙げる、解釈する、要約する

例	備考
機能正確性、機能適切性、機能完全性テストを区別する。	コンセプトの違いを探す。
CRUDテストを説明する。	
テスト環境要件の例を挙げる。	
さまざまなソフトウェア開発ライフサイクルにおけるテストアナリストの関わりについてまとめることができる。	

レベル3: 応用 (K3)

身近な課題に直面したときに手順を実行できる、または正しい手順を選択し、与えられた状況に適用できる。

動作動詞：適用する、実施する、準備する、使用する

例	備考
ドメインテストを適用する。	手順／技術／プロセスなどを指す。
セッションベースドテストのテストチャーターを準備する。	
キーワード駆動テストを使用してテストスクリプトを開発する。	受験者に技術や手順を使用できるようになること求めるLOで使用できる。「適用する」に近い。

レベル4: 分析する（K4）

©International Software Testing Qualifications Board

手順や技術に関連する情報を、理解を深めるために構成要素に分離し、事実と推論を区別することができる。典型的な用途は、文書、ソフトウェア、プロジェクトの状況を分析し、問題やタスクを解決するための適切な行動を提案することである。

動作動詞：分析する、分解する、概要を描く、優先順位を付ける、選択する。

例	備考
変更の影響を解析し、リグレッションテストの範囲を決定する。	測定可能な分析目標と組み合わせてのみ審査可能である。「XをYに分析する」(または類似の)形式でなければならない。
状況に応じて、プロダクトリスクを軽減するための適切なテスト技法を選択する。	選択が分析を必要とする場合には要件となる。

参考

(学習の目的の認知レベルについて)

Anderson, L. W. and Krathwohl, D. R. (eds) (2001) A Taxonomy for Learning, Teaching, and Assessing：A Revision of Bloom's Taxonomy of Educational Objectives, Allyn & Bacon.

©International Software Testing Qualifications Board

8 付録B-学習の目的とビジネス成果のトレーサビリティマトリクス

ここでは、ビジネス成果とAdvancedLevelテストアナリストの学習目的とのトレーサビリティを示す。

ビジネス成果：AdvancedLevelテストアナリスト		TA-BO1	TA-BO2	TA-BO3	TA-BO4	TA-BO5	TA-BO6	TA-BO7	TA-BO8	TA-BO9
TA-BO1	ソフトウェア開発ライフサイクルに基づいた適切なテストを支援し、実施する。	6								
TA-BO2	リスクベースドテストの原則を適用する。		3							
TA-BO3	テスト目的の達成を支援するために、適切なテスト技法を選択し、適用する。			13						
TA-BO4	適切な詳細度と品質の文書を提供する。				5					
TA-BO5	実施すべき機能テストの適切な種類を決定する。					1				
TA-BO6	非機能テストに貢献する。						3			
TA-BO7	欠陥予防に貢献する。							5		
TA-BO8	ツールの活用により、テストプロセスの効率を改善する。								4	
TA-BO9	テスト環境とテストデータの要件を明確にする。									2

©International Software Testing Qualifications Board

ビジネス成果：AdvancedLevelテストアナリスト		TA-BO1	TA-BO2	TA-BO3	TA-BO4	TA-BO5	TA-BO6	TA-BO7	TA-BO8	TA-BO9
LO番号	学習の目的（Kレベル）									
1	テストプロセスにおけるテストアナリストのタスク - 225分									
1.1	ソフトウェア開発ライフサイクルにおけるテスト									
TA-1.1.1	さまざまなソフトウェア開発ライフサイクルにおけるテストアナリストの関わりについて要約する。 (K2)	×								
1.2	テスト活動への関与									
TA-1.2.1	テスト分析の一環としてテストアナリストが行うタスクを要約する。（K2）	×								
TA-1.2.2	テスト設計の一環としてテストアナリストが行うタスクを要約する。（K2）	×								
TA-1.2.3	テスト実装の一環としてテストアナリストが行うタスクを要約する。（K2）	×								
TA-1.2.4	テスト実行の一環としてテストアナリストが行うタスクを要約する。（K2）	×								
1.3	作業成果物に関するタスク									
TA-1.3.1	ハイレベルテストケースとローレベルテストケースを区別する。 (K2)				×					
TA-1.3.2	テストケースの品質基準について説明する。（K2）				×					
TA-1.3.3	テスト環境要件の例を挙げる。（K2）				×					×
TA-1.3.4	テストオラクル問題とその解決策を説明する。(K2)			×	×					
TA-1.3.5	テストデータ要件の例を挙げる。（K2）				×					×
TA-1.3.6	キーワード駆動テストを使用してテストスクリプトを開発する。（K3）	×							×	
TA-1.3.7	テストウェアをマネジメントするツールの種類を要約する。(K2)								×	
2	リスクベースドテストにおけるテストアナリストのタスク - 90分									

©International Software Testing Qualifications Board

ビジネス成果：AdvancedLevelテストアナリスト		TA-BO1	TA-BO2	TA-BO3	TA-BO4	TA-BO5	TA-BO6	TA-BO7	TA-BO8	TA-BO9
2.1	リスク分析									
TA-2.1.1	プロダクトリスク分析におけるテストアナリストの貢献を要約する。（K2）		×							
2.2	リスクコントロール									
TA-2.2.1	リグレッションテストの範囲を決定するために変更の影響を分析する。（K4）		×						×	
3	テスト分析とテスト設計 - 615分									
3.1	データに基づくテスト技法									
TA-3.1.1	ドメインテストを適用する(K3)			×						
TA-3.1.2	組み合わせテストを適用する(K3)			×						
TA-3.1.3	ランダムテストの利点と制限についてまとめる（K2）。			×						
3.2	振る舞いに基づくテスト技法									
TA-3.2.1	CRUDテストについて説明する（K2）			×						
TA-3.2.2	状態遷移テストを適用する(K3)			×						
TA-3.2.3	シナリオベースドテストを適用する（K3）			×						
3.3	ルールに基づくテスト技法									
TA-3.3.1	デシジョンテーブルテストを適用する(K3)			×						
TA-3.3.2	メタモルフィックテストを適用する(K3)			×						
3.4	経験ベースのテスト									
TA-3.4.1	セッションベースドテストのテストチャーターを準備する。（K3）			×						
TA-3.4.2	経験ベースのテストを支援するチェックリストを準備する。(K3)			×						

©International Software Testing Qualifications Board

ビジネス成果：AdvancedLevelテストアナリスト		TA-BO1	TA-BO2	TA-BO3	TA-BO4	TA-BO5	TA-BO6	TA-BO7	TA-BO8	TA-BO9
TA-3.4.3	クラウドテストの利点と制限の例を挙げる（K2）。			×						
3.5	最適なテスト技法を適用する									
TA-3.5.1	ある状況に対してプロダクトリスクを軽減する適切なテスト技法を選択する。（K4）		×	×						
TA-3.5.2	テスト設計の自動化の利点とリスクを説明する。（K2）								×	
4	品質特性のテスト - 60分									
4.1	機能テスト									
TA-4.1.1	機能正確性、機能適切性、機能完全性テストを区別する。（K2）					×				
4.2	使用性テスト									
TA-4.2.1	テストアナリストが使用性テストにどのように貢献するかを説明する。（K2）						×			
4.3	柔軟性テスト									
TA-4.3.1	テストアナリストが適応性テストおよびインストール性テストにどのように貢献するかを説明する。（K2）						×			
4.4	互換性テスト									
TA-4.4.1	テストアナリストが相互運用性テストにどのように貢献するかを説明する。（K2）						×			
5	ソフトウェア欠陥予防 - 225分									
5.1	欠陥予防の実践									
TA-5.1.1	テストアナリストが欠陥予防にどのように貢献できるかを説明する。（K2）							×		
5.2	フェーズ内封じ込めの支援									
TA-5.2.1	仕様書の欠陥を検出するためにテスト対象のモデルを使用する 。(K3)							×		

©International Software Testing Qualifications Board

ビジネス成果：AdvancedLevelテストアナリスト		TA-BO1	TA-BO2	TA-BO3	TA-BO4	TA-BO5	TA-BO6	TA-BO7	TA-BO8	TA-BO9
TA-5.2.2	欠陥を発見するためにレビュー技法をテストベースに適用する。(K3)							×		
5.3	類似欠陥の再発の軽減									
TA-5.3.1	欠陥検出に関する改善の可能性を識別するためにテスト結果を分析する。（K4）							×		
TA-5.3.2	欠陥分類が根本原因分析にどのように役立つかを説明する。（K2）							×		

©International Software Testing Qualifications Board

9 付録C - リリースノート

ISTQB® Advanced Level TestAnalyst Syllabus v4.0 はメジャーアップデートである。そのため、章や節ごとの詳細なリリースノートはない。しかし、主な変更点の要約を以下に示す。さらに、ISTQB®は、別のリリースノート文書において、Advanced Level テストアナリストシラバス v3.1.2 と v4.0 バージョンの「学習の目的」の詳細なトレーサビリティを提供している。

今回のメジャーリリースでは、以下の変更が加えられた:

シラバスの構成
すべての学習の目的は、それぞれが単一の要素(不可分な単位)となるよう編集されている。また、対応する学習の目的を持たないコンテンツが生じないよう、学習の目的からコンテンツへの1対1のトレーサビリティが確立されている。各学習の目的は、同じ番号を持つ節や項で説明されている(例えば、TA-1.1.1は1.1.1項で説明されている)。本バージョンのゴールは、読みやすさ、理解しやすさ、学習しやすさ、翻訳しやすさを向上させることであり、実践的な有用性の向上と、知識およびスキルのバランスを重視している。

v3.1.2では31個だった学習の目的は、v4.0では36個になった。いくつかのK4の学習の目的は、K2またはK3に引き下げられている。テスト技法に関する学習の目的については、さらなるテスト設計に向けて文書を分析することではなく、テスト技法を適用することに重点を置くべきだというのが、変更の理由であった。また、いくつかの学習の目的は1つに統合した。下の表は、学習の目的の数とトレーニング時間に関する詳細な統計を示している。

シラバス	#K2 LO	#K3 LO	#K4 LO	#合計LO
本Ver. (4.0)	22 (330分)	11 (660分)	3 (225分)	36 (1215分)
前Ver. (3.1.2)	16 (240分)	5 (300分)	10 (750分)	31 (1290分)

テスト技法の分類
第3章の構成は、v3.1.2と比較して、テスト技法のより詳細な分類を反映している。ブラックボックステスト技法は、基礎となるテスト対象モデルのタイプに従って、データに基づく技法、振る舞いに基づく技法、ルールに基づく技法に分類するようになった。

第5章の範囲を拡大
旧第5章(レビューのみに特化)は、仕様書の欠陥を検出するためのモデルの使用、欠陥検出を改善するためのテスト結果の分析、根本原因分析を支援するための欠陥分類の使用など、TAが使用する他の本質的な欠陥予防の実践方法について説明するために拡張している。

学習の目的の変更
- 第1章は、テスト活動に関する節とテストウェアに関する節に再編成している。
- ハイレベルテストケースとローレベルテストケースに関するトピック(旧TA-1.4.2)は、K4からK2(TA-1.3.1)に格下げされた。
- リスクベースドテストに関するK3のLO(旧TA-2.1.1)は、リスク分析に関するK2 TA-2.1.1とリスクコントロールに関するK4 TA-2.2.1に分割された。
- 同値分割法(旧K4 TA-3.2.1)と境界値分析(旧K4 TA-3.2.2)は、領域テストに関するより一般的なK3 TA-3.1.1に置き換えられた。
- ペアワイズテスト(旧K4 TA-3.2.6)とクラシフィケーションツリーダイアグラム(旧K2 TA-3.2.5)は、組み合わせテストに関するより一般的なK3 TA-3.1.2に統合した。

©International Software Testing Qualifications Board

- 状態遷移テスト（旧 K4 TA-3.2.4）は、K3 TA-3.2.2に置換され、Nスイッチとラウンドトリップカバレッジに重点を置いた。CTFL との重複は削除した。
- ユースケーステスト（旧K4 TA-3.2.7）は、シナリオベースドテストに関するより一般的なK3 TA-3.2.3に置換した。
- デシジョンテーブルテスト（旧 K4 TA-3.2.3）は、K3（TA-3.3.1）に格下げした。
- 探索的テスト（旧 K3 TA-3.3.2）は、テストチャーターの作成（K3 TA-3.4.1）と経験ベースのテストを支援するチェックリストの作成（K3 TA-3.4.2）の 2 つの別々の LO に置換された。現行の CTFL v4.0 との重複は削除した。
- テスト技法の比較と最適な技法の適用に関する4つのLO（旧：K4 TA-3.2.8、K2 TA-3.3.3、K2 TA-3.4.1、K2 TA-3.3.1）は、K4 TA-3.5.1に統合した。
- 機能テストに関する4つのLO（旧：K2 TA-4.2.1、K2 TA-4.2.2、K2 TA-4.2.3、K4 TA-4.2.7）はK2 TA-4.1.1に統合した。機能テストはすでに第3章のテスト技法の文脈で大部分説明されているため、このトピックは簡略化した。
- 第6章（テストツール）のトピックは、より実践的な問題に焦点を当て、第1.3節に移した。旧 K3 TA-6.2.1『特定のシナリオで、キーワード駆動テストプロジェクトでのテストアナリストの適切な活動を判断する』は、K3 TA-1.3.6『キーワード駆動テストを使用してテストスクリプトを開発する』に若干変更した。
- 旧 K2 TA-6.3.1『テスト設計、テストデータ準備、テスト実行で適用するテストツールの使用方法と種類を説明する』は、K2 TA-1.3.7『テストウェアをマネジメントするツールの種類を要約する』に若干変更した。
- レビューに関する2つの類似したLO（旧K3 TA-5.2.1とK3 TA-5.2.2）は、1つのK3 TA-5.2.2に統合した。

新しいトピック

- テストケースの品質基準（K2 TA-1.3.2）
- テスト環境要件 (K2 TA-1.3.3)
- テストオラクルの決定 (K2 TA-1.3.4)
- テストデータ要件 (K2 TA-1.3.5)
- ランダムテスト（K2 TA-3.1.3）
- CRUD テスト (K2 TA-3.2.1)
- メタモルフィックテスト (K3 TA-3.3.2)
- クラウドテスト（K2 TA-3.4.3）
- テスト設計の自動化の利点とリスク（K2 TA-3.5.2）
- テストアナリストの欠陥予防への貢献（K2 TA-5.1.1）
- 仕様書の欠陥を検出するためにテスト対象のモデルを使用する（K3 TA-5.2.1）
- 欠陥検出を改善するためのテスト結果の分析（K4 TA-5.3.1）
- 欠陥分類による根本原因分析の支援 (K2 TA-5.3.2)

スタンダード・アップデート

ソフトウェア品質に関する国際規格（ISO/IEC 25010, 2023）およびテスト技法に関する国際規格（ISO/IEC/IEEE 29119-4, 2021）の最新バージョンの変更に伴い、シラバスの対応内容も変更した。

©International Software Testing Qualifications Board

10 付録D - 略語リスト

略語	意味
AI	人工知能
BPMN	ビジネスプロセスモデル表記法
BVA	境界値分析
CRUD	作成、読み込み、更新、削除
CSV	カンマ区切り値
DDP	欠陥検出率
DRE	欠陥除去効率
EP	同値分割法
GDPR	一般データ保護規則
JSON	JavaScriptオブジェクト記法
MBT	モデルベースドテスト
MR	メタモルフィック関係
MT	メタモルフィックテスト
ODC	直交欠陥分類
PCE	フェーズ内封じ込め有効性
RCA	根本原因分析
SDLC	ソフトウェア開発ライフサイクル
TA	テストアナリスト
TTA	テクニカルテストアナリスト
UML	統一モデリング言語
UX	ユーザーエクスペリエンス
XML	拡張マークアップ言語

©International Software Testing Qualifications Board

11 付録E - 特定の専門分野（ドメイン）特有の用語

期間	定義
CRUDマトリクス	システム内のエンティティに対する機能のアクションタイプを示すマトリクス。
データセマンティクス	データの意味と解釈。
なぜなぜ分析	欠陥や問題の根本原因を特定するために、「なぜ」という質問を5回繰り返し、その都度、現在の「なぜ」を前の「なぜ」の答えに導く反復質問法。
パレート分析	最大の効果が期待できる問題領域またはタスクを識別するアプローチ。
ユーザージャーニーマップ	ユーザーエクスペリエンスを視覚化した文書で、ユーザーがゴールを達成するために取るプロセスのステップを示す。
ユーザーリサーチ	ユーザーがどのようにタスクを実行するかを研究したり、ユーザーがコンポーネントやシステムとどのように相互作用するかを観察したり、あるいはデータ分析や解釈によって、ユーザーのニーズや思考プロセスを学ぶ専門分野。

©International Software Testing Qualifications Board

12 付録F - ソフトウェア品質モデル

下表は、ISO/IEC 25010 製品品質モデル(ISO/IEC 25010, 2023)の品質特性を示している。本シラバス(TA)の中でどの特性／副特性が扱われ、他の ISTQB®シラバス(テクニカルテストアナリスト(TTA)、性能テスト担当者(PT)、使用性テスト担当者(UT)、自動車ソフトウェアテスト担当者(AuT)、セキュリティテスト担当者(SEC))でどの特性が扱われているかを指標で示している。複数のシラバスがある品質特性をカバーしている場合は、最も詳細にカバーしているものを最初に記載している。また、この表では、現在の ISO/IEC 25010 モデルと、本シラバスの前バージョンで使用された2011年版とを比較している。

ISO/IEC 25010:2023 (現バージョン)	**ISO/IEC 25010:2011 (前バージョン)**	**備考**	**ISTQB® シラバス**
機能適合性	**機能適合性**		**TA**
機能完全性	機能完全性		
機能正確性	機能正確性		
機能適切性	機能適切性		
性能効率性	**性能効率性**		PT、TTA
時間効率性	時間効率性		
資源効率性	資源効率性		
容量満足性	容量満足性		
互換性	**互換性**		**TA**、TTA
共存性	共存性		TTA
相互運用性	相互運用性		**TA**
インタラクション容易性（対話性）	**使用性**	**名称変更**	UT、**TA**
適切度認識性	適切度認識性		
習得性	習得性		
運用操作性	運用操作性		
ユーザエラー防止性	ユーザエラー防止性		
ユーザーエンゲージメント（利用者関与性）	ユーザーインタフェース快美性	名称変更	
インクルーシビティ（包摂性）	アクセシビリティ	分割と名称変更	
ユーザー支援性			
自己記述性		新規	

©International Software Testing Qualifications Board

ISO/IEC 25010:2023 (現バージョン)	ISO/IEC 25010:2011 (前バージョン)	備考	ISTQB® シラバス
信頼性	**信頼性**		TTA
無障害性	成熟性	名称変更	
可用性（アベイラビリティ）	可用性		
障害許容性（耐故障性）	障害許容性		
回復性	回復性		
セキュリティ	**セキュリティ**		SEC、TTA
機密性	機密性		
インテグリティ	インテグリティ		
否認防止性	否認防止性		
責任追跡性	責任追跡性		
真正性	真正性		
耐攻撃性		新規	
保守性	**保守性**		TTA
モジュール性	モジュール性		
再利用性	再利用性		
解析性	解析性		
修正性	修正性		
テスト可能性（試験性）	試験性		
柔軟性	**移植性**	**名称変更**	TTA、**TA**、PT
適応性	適応性		**TA**、TTA
拡張性		新規	PT
インストール性（設置性）	設置性		**TA**、TTA
置換性	置換性		TTA
安全性		新規	AuT
運用制約性		新規	
リスク識別性		新規	
フェールセーフ		新規	
危険警告性		新規	
安全統合性		新規	

©International Software Testing Qualifications Board

13 付録G - 商標

ISTQB® は、International Software Testing Qualifications Board の登録商標である。

UML® は、OMG（Object Management Group）の登録商標である。

BPMN™はOMG（Object Management Group）の商標である。

©International Software Testing Qualifications Board